APPLICATION NOTE

ATBM WIFI6+BLE 驱动配置说明_FAQ

ATBM606X

1x1 802.11b/g/n/ax
Wi-Fi+BLE 芯片

Table of contents

1 简介

2 单独编译驱动的编译方法

2.1 驱动通过 Make menuconfig 进行配置相关参数
(1) 配置编译环境
(2) 进到驱动根目录执行，make menuconfig
2.2 配置说明
(1) 选择 wifi 芯片型号
(2) 选择通信总线接口
(3) 选择固件方式
2.2.3.1 使用 bin 文件
2.2.3.2 使用.h 文件
(4) 驱动一些扩展功能
(5) 内部调试使用
(6) 设置默认国家码
(7) 修改 wifi 接口名称
(8) 修改驱动名称以及挂载结点名称
2.2.8.1 注意【重要】
(9) 配置驱动自带的 CFG80211 功能
(10) 自定义 ATBM WIFI 驱动支持 WPA3
(11) BLE 支持
2.2.11.1 WIFI + BLE 连接共存配置
2.2.11.2 增加一个 WIFI+BLE 过认证的固件
2.2.11.3 去掉纯 WIFI 的固件
2.3 SDIO WIFI 移植配置说明
(1) 注意
(2) SDIO 中断方式
2.3.2.1 修改驱动根目录的 Makefile
2.3.2.2 修改 hal_apollo/apollo_plat.h

AltoBeam Inc. Confidential Document

2.3.2.3 定义 ATBM_WIFI_PLATFORM 的值用于全局 24
2.3.2.4 修改 hal_apollo/atbm_platform.c 25
(3) GPIO 中断方式 25
2.3.3.1 修改驱动根目录的 Makefile 25
2.3.3.2 打开支持 GPIO 中断配置 26
2.3.3.3 修改使用平台的 mmc 口 26
2.3.3.4 修改 hal_apollo/apollo_plat.h 26
2.3.3.5 修改 hal_apollo/atbm_platform.c 27
(4) 复位&扫卡动作 28
2.3.4.1 注册 28
2.3.4.2 复位 29
2.3.4.3 扫卡 29
2.4 编译 30
(1) 直接编译 30
(2) 带参数编译 30
(3) 使用 build_driver.sh 脚本编译 31
2.5 驱动所在的目录 32
3 驱动放置在内核中的编译方法 33
3.1 将驱动放置在内核中 33
4 出错调试信息&解决 34
4.1 Make menuconfig 报错 34
4.2 编译出错 35
4.3 加载出错 36
(1) NO_CONFIRM 宏没配置对导致出错 36
4.4 扫描 AP 个数少 36
(1) 扫描状态返回-110 36
(2) 扫描状态正常但是扫描的 AP 数量少，并且发现前几个信道的 ap 很少或者没有 37
4.5 添加详细的反汇编信息方法 37
4.6 编译的时候显示详细的编译信息 38
4.7 编译报时钟错误 38
4.8 编译报驱动自带的 cfg80211.ko 里面有一些 undefined 39
4.9 编译驱动报 cfg80211 函数 export twice Previous export was in vmlinux 39
4.10 Linux 3.10 内核同时跑 ap+sta 模式失败 40
4.11 富瀚平台编译 ATBM WIFI 驱动会报一些函数 undefined 40
4.12 扫描不到某些信道的 AP 41
(1) 扫不到低信道的 ap 41
(2) 扫不到高信道的 AP 42

AltoBeam Inc. Confidential Document

作者	版本	说明
Yuzhihuang	V1.0	Init
Yuzhihuang	V1.1	修改配置说明
Yuzhihuang	V1.2	添加 ble 配置项以及编译方法
Yuzhihuang	V1.3	添加出错调试信息&解决方法
Yuzhihuang	V1.4	拿掉 CMD 聚合配置项
Yuzhihuang	V1.5	1、添加 usb urb no interrupt 配置说明 2、添加默认国家码配置 3、添加 WIFI & BLE 共存配置说明
Yuzhihuang	V1.6	1、增加 build_driver.sh 使用说明
Yuzhihuang	V1.7	修改 BLE 配置说明
Yuzhihuang	V1.8	添加 h 文件说明
Yuzhihuang	V1.9	添加 bin 文件说明
Yuzhihuang	V1.10	修改 sdio platform 配置说明，修复原本配置会导致内核整编慢的问题
Yuzhihuang	V1.11	
Yuzhihuang	V1.12	修改自定义 firmware 路径
Eason li	V1.13	增加 WiFi6 最新芯片配置和对应的 firmware

AltoBeam Inc. Confidential Document

1 简介

Altobeam Wi-Fi 6 驱动支持 USB 和 SDIO 接口，支持 Linux Kernel 3.0 ~ 5.10 版本。

Wi-Fi USB 接口型号芯片驱动编译会相对简单，编译步骤需配置工具链、内核路径、Wi-Fi 型号及 USB 总线、选择功能支持，编译生成驱动后在板子上都能跑起来。

Wi-Fi SDIO 接口型号芯片驱动编译相对复杂，除了上面 USB 编译步骤外，还需要清楚 Soc 内核 SDIO 识卡接口、是否支持 SDIO 中断、控制 Wi-Fi 芯片复位接口，整个流程相对复杂，请按照文档中 SDIO WIFI 移植配置说明章节内容修改。

2 单独编译驱动的编译方法

2.1 驱动通过 Make menuconfig 进行配置相关参数

(1) 配置编译环境

修改驱动根目录的 Makefile,配置对应的内核路径，平台架构以及交叉编译工具链

22684 以前的版本编译配置：

```
#PLATFORM_HI3516EV200                    19
#PLATFORM_XUNWEI_2G                      20
#PLATFORM_NVT98517      21
export
platform ?= PLATFORM_SIGMASTAR
#Android
#Linux
sys ?= linux
#arch:arm or arm64 or mips(NVT98517)
arch ?= arm
#export
#ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=$(platform)

ifeq ($(CUSTOMER_SUPPORT_USED),y)
MAKEFILE_SUB ?= Makefile.build.customer
else
MAKEFILE_SUB ?= Makefile.build
endif

ifeq ($(platform),PLATFORM_SIGMASTAR)
KERDIR:=/usr/lchome/yuzhihuang/Mstar/325/kernel
CROSS_COMPILE:=/usr/lchome/yuzhihuang/Mstar/325/arm-buildroot-linux-uclibcgnueabihf-4.9.4/bin/arm-buildroot-linux-uclibcgnueabi
hf-
arch = arm
ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=18
```

22684 以后的版本配置：

```
35:
36:
37: platform ?= PLATFORM_SIGMASTAR
38: #ATBM_BUILD_IN_KERNEL?=
39:
40:
41: ifeq ($(KERNELRELEASE),)
42: export
43: ATBM_BUILD_IN_KERNEL=n
44:
45: ifeq ($(CUSTOMER_SUPPORT_USED),y)
46: MAKEFILE_SUB ?= Makefile.build.customer
47: else
48: MAKEFILE_SUB ?= Makefile.build
49: endif
50:
51:
52: KERDIR ?= /usr/lchome/yuzhihuang/Mstar/IPC_I3/linux3.18_i3/
53: CROSS_COMPILE ?= /usr/lchome/yuzhihuang/Mstar/IPC_I3/arm-linux-gnueabihf-4.8.3-201404/bin/arm-linux-gnueabihf-
54: #Android
55: #Linux
56: sys ?= Linux
57: #arch:arm or arm64 or mips(NVT98517)
58: arch ?= arm
59:
```

Altobeam Confidential Document

参数说明：

platform ： 配置平台，客户自己设置，平台设置对应的值需要按照如下累加，

需要通过这个方式配置将平台对应的值在驱动中生效：

```
export ATBM_WIFI__EXT_CCFLAGS ?= -DATBM_WIFI_PLATFORM=18
```

```
#PLATFORM_XUNWEI                 1
#PLATFORM_SUN6I                      2
#PLATFORM_FRIENDLY               3
#PLATFORM_SUN6I_64               4
#PLATFORM_SUN8I                      5
#PLATFORM_HI3518E                6
#PLATFORM_X86PC                  7
#PLATFORM_AMLOGIC                8
#PLATFORM_AMLOGIC_905            8
#PLATFORM_AMLOGIC_905X               9
#PLATFORM_ROCKCHIP               10
#PLATFORM_MSTAR                      11
#PLATFORM_CDLINUX                12
#PLATFORM_AMLOGIC_S805               13
#PLATFORM_HIS_LINUX_3_4              14
#PLATFORM_ROCKCHIP_3229              15
#PLATFORM_ROCKCHIP_3229_ANDROID8 16
#PLATFORM_HS_IPC                 17
#PLATFORM_SIGMASTAR                18
#PLATFORM_HI3516EV200               19
#PLATFORM_XUNWEI_2G                20
#PLATFORM_NVT98517               21
#PLATFORM_INGENICT31             22
#PLATFORM_INGENICT41             23
```

KERDIR ： 内核路径

CROSS_COMPILE ： 交叉编译器

sys ： 当前运行的系统，有 Linux ， Android

arch ： 平台架构，arm,arm64,mips 等

最后在 Makefile 末尾添加上平台对应的 ID

AltoBeam Inc. Confidential Document

```
ATBM_WIFI__EXT_CCFLAGS ?= -DATBM_WIFI_PLATFORM=8
endif
ifeq ($(platform),PLATFORM_CDLINUX)
export
ATBM_WIFI__EXT_CCFLAGS ?= -DATBM_WIFI_PLATFORM=12
endif
ifeq ($(platform),PLATFORM_AMLOGIC_S805)
export
ATBM_WIFI__EXT_CCFLAGS ?= -DATBM_WIFI_PLATFORM=13
endif

ifeq ($(platform),PLATFORM_SIGMASTAR)
export
ATBM_WIFI__EXT_CCFLAGS ?= -DATBM_WIFI_PLATFORM=18
endif

ifeq ($(platform),PLATFORM_INGENICT31)
export
ATBM_WIFI__EXT_CCFLAGS ?= -DATBM_WIFI_PLATFORM=22
endif
ifeq ($(platform),PLATFORM_INGENICT41)
export
ATBM_WIFI__EXT_CCFLAGS ?= -DATBM_WIFI_PLATFORM=23
endif


ifeq ($(ATBM_BUILD_IN_KERNEL),)

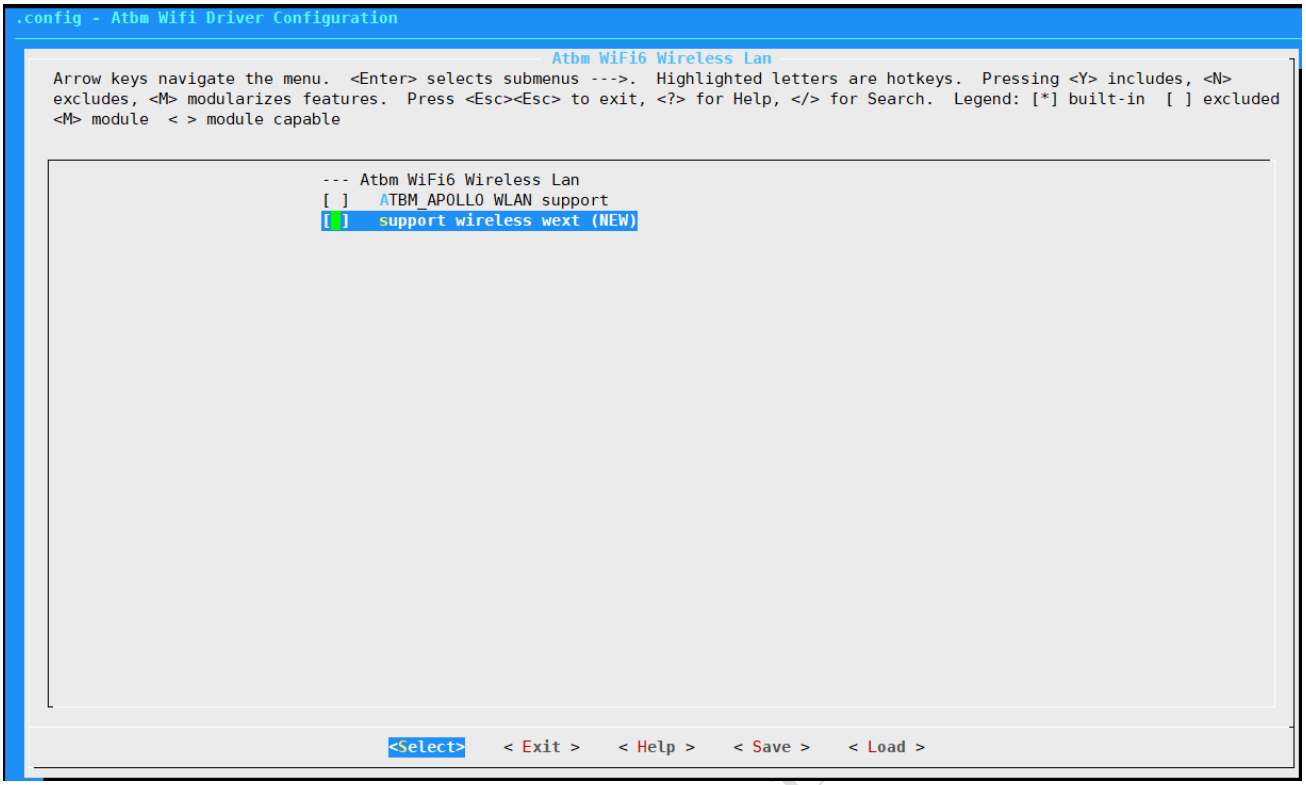
obj-m += hal_apollo/
export ATBM_BUILD_IN_KERNEL=y

else

export
include $(src)/Makefile.build.kernel
```

（2） 进到驱动根目录执行，make menuconfig

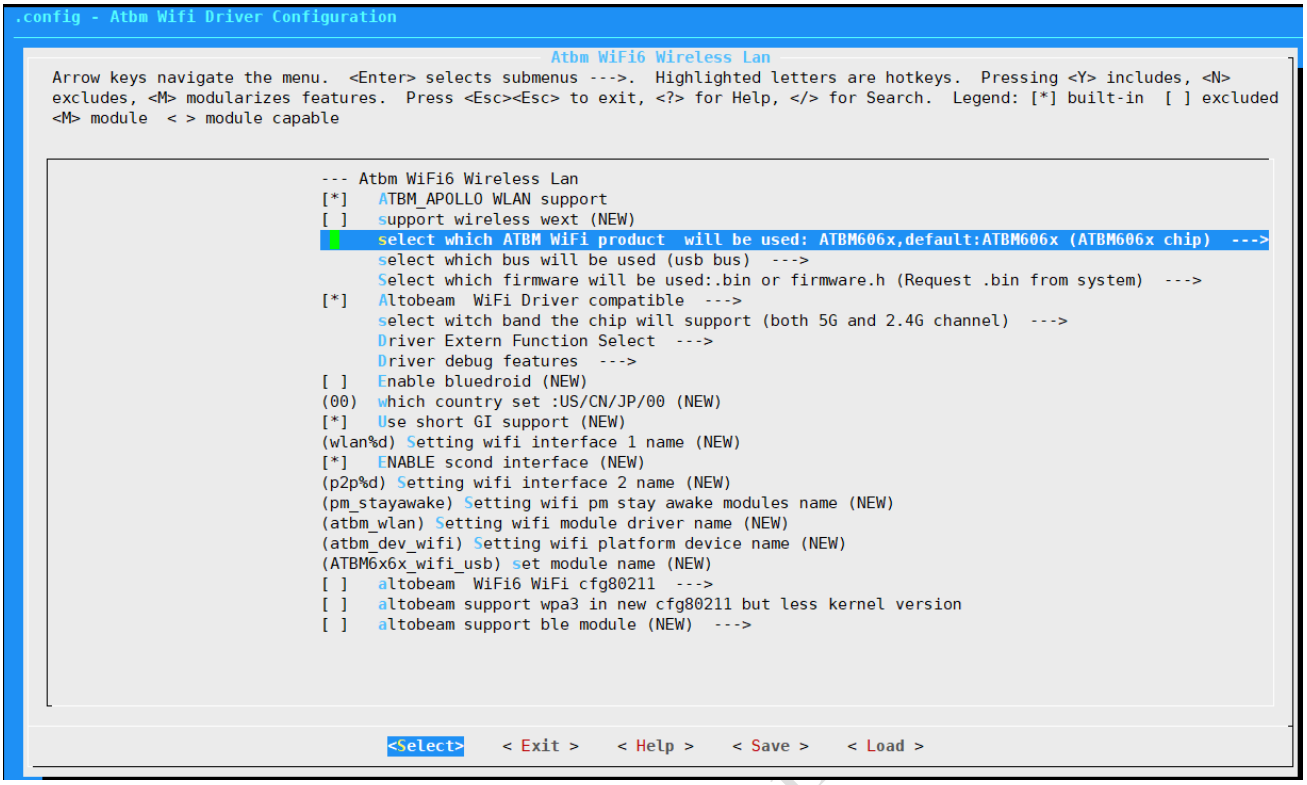
AltoBeam Inc. Confidential Document

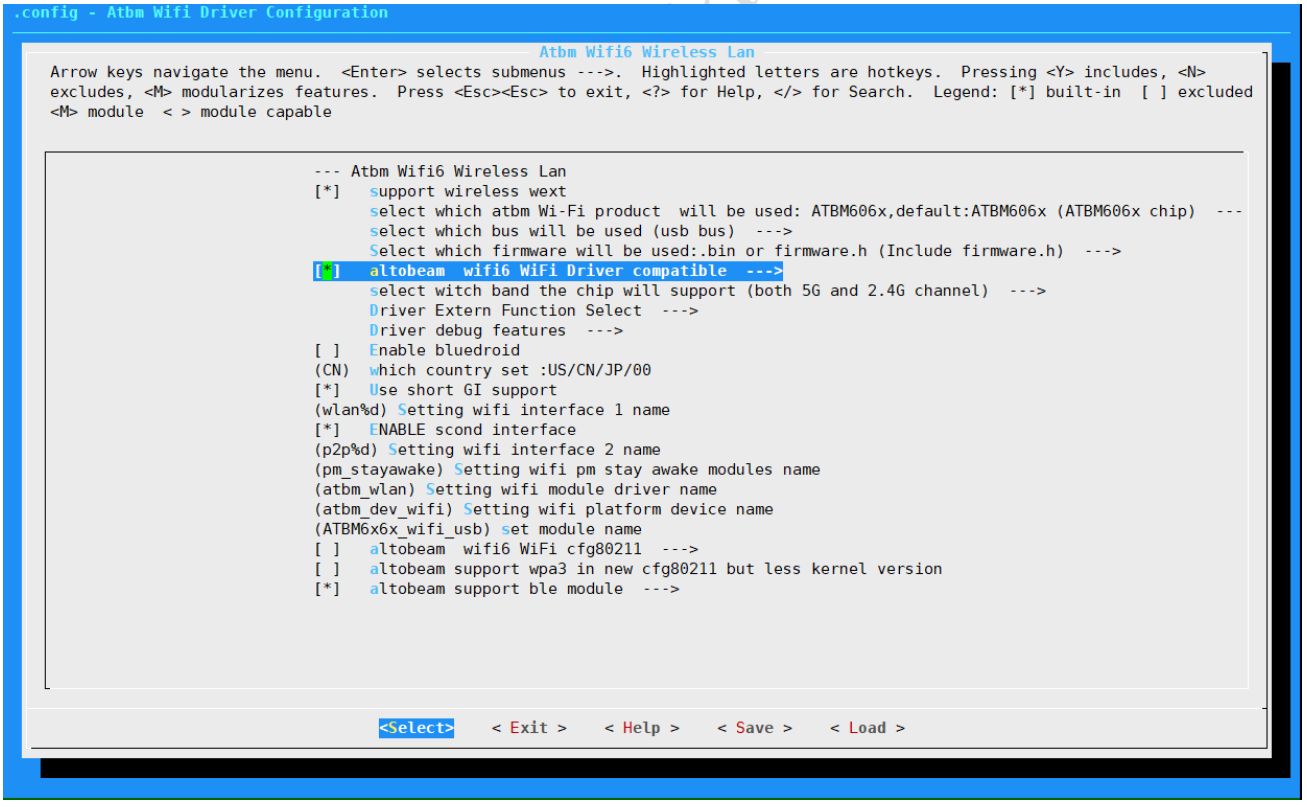


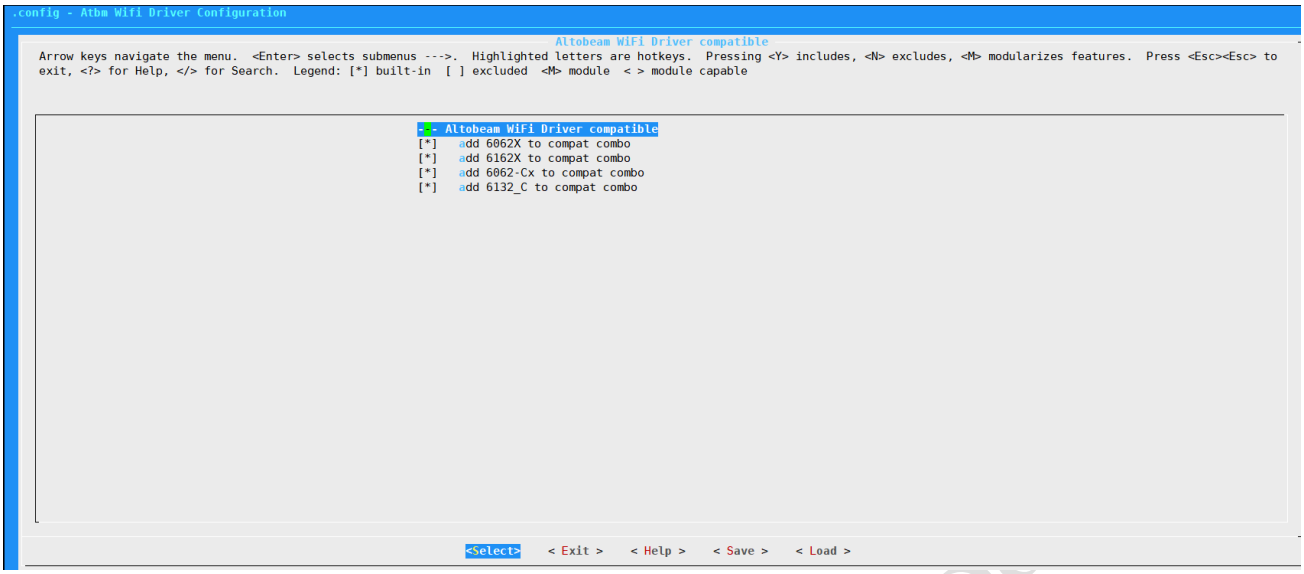
2.2 配置说明

选择 **ATBM WiFi** 芯片型号，当前驱动兼容 ATBM6062 、ATBM6062C、 ATBM6162、ATBM6132C。

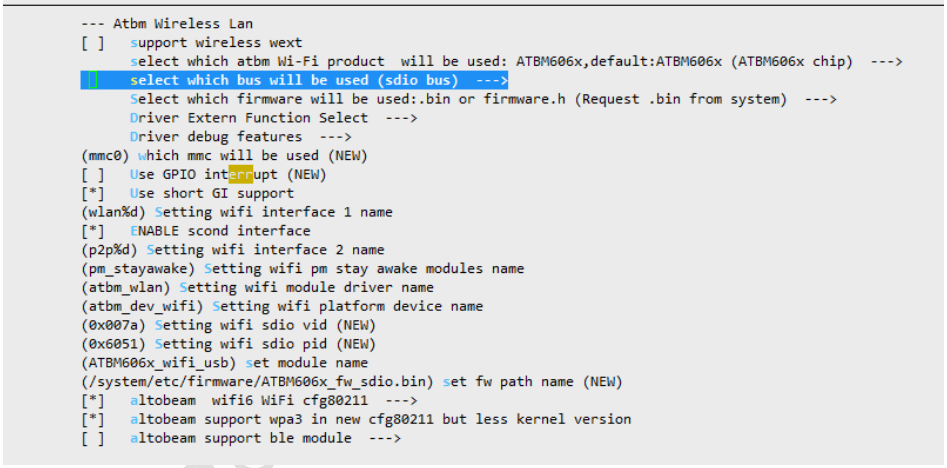
AltoBeam Inc. Confidential


.config - Atbm Wifi Driver Configuration
Atbm WiFi6 Wireless Lan
Arrow keys navigate the menu. <Enter> selects submenus --->. Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in [ ] excluded <M> module < > module capable
--- Atbm WiFi6 Wireless Lan
[*] ATBM_APOLLO WLAN support
[ ] support wireless wext (NEW)
select which ATBM WiFi product will be used: ATBM606x,default:ATBM606x (ATBM606x chip) --->
select which bus will be used (usb bus) --->
Select which firmware will be used:.bin or firmware.h (Request .bin from system) --->
[*] Altobeam WiFi Driver compatible --->
select witch band the chip will support (both 5G and 2.4G channel) --->
Driver Extern Function Select --->
Driver debug features --->
[ ] Enable bluedroid (NEW)
(00) which country set :US/CN/JP/00 (NEW)
[*] Use short GI support (NEW)
(wlan%d) Setting wifi interface 1 name (NEW)
[*] ENABLE scond interface (NEW)
(p2p%d) Setting wifi interface 2 name (NEW)
(pm_stayawake) Setting wifi pm stay awake modules name (NEW)
(atbm_wlan) Setting wifi module driver name (NEW)
(atbm_dev_wifi) Setting wifi platform device name (NEW)
(ATBM6x6x_wifi_usb) set module name (NEW)
[ ] altobeam WiFi6 WiFi cfg80211 --->
[ ] altobeam support wpa3 in new cfg80211 but less kernel version
[ ] altobeam support ble module (NEW) --->
<Select> < Exit > < Help > < Save > < Load >

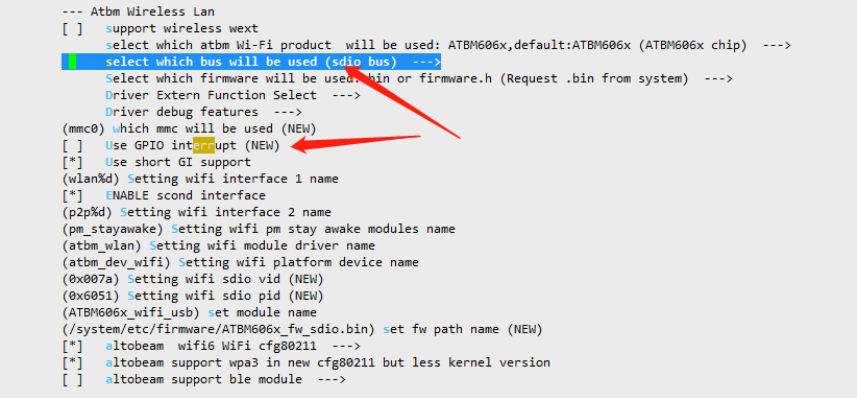

.config - Atbm Wifi Driver Configuration
Atbm Wifi6 Wireless Lan
Arrow keys navigate the menu. <Enter> selects submenus --->. Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in [ ] excluded <M> module < > module capable
--- Atbm Wifi6 Wireless Lan
[*] support wireless wext
select which atbm Wi-Fi product will be used: ATBM606x,default:ATBM606x (ATBM606x chip) ---
select which bus will be used (usb bus) --->
Select which firmware will be used:.bin or firmware.h (Include firmware.h) --->
[*] altobeam wifi6 WiFi Driver compatible --->
select witch band the chip will support (both 5G and 2.4G channel) --->
Driver Extern Function Select --->
Driver debug features --->
[ ] Enable bluedroid
(CN) which country set :US/CN/JP/00
[*] Use short GI support
(wlan%d) Setting wifi interface 1 name
[*] ENABLE scond interface
(p2p%d) Setting wifi interface 2 name
(pm_stayawake) Setting wifi pm stay awake modules name
(atbm_wlan) Setting wifi module driver name
(atbm_dev_wifi) Setting wifi platform device name
(ATBM6x6x_wifi_usb) set module name
[ ] altobeam wifi6 WiFi cfg80211 --->
[ ] altobeam support wpa3 in new cfg80211 but less kernel version
[*] altobeam support ble module --->
<Select> < Exit > < Help > < Save > < Load >


.config - Atbm Wifi Driver Configuration
Altobeam WiFi Driver compatible
Arrow keys navigate the menu. <Enter> selects submenus --->. Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in [ ] excluded <M> module < > module capable
-*- Altobeam WiFi Driver compatible
[*] add 6062X to compat combo
[*] add 6162X to compat combo
[*] add 6062-Cx to compat combo
[*] add 6132_C to compat combo
<Select> < Exit > < Help > < Save > < Load >

（1） 选择通信总线接口


--- Atbm Wireless Lan
[ ] support wireless wext
select which atbm Wi-Fi product will be used: ATBM606x,default:ATBM606x (ATBM606x chip) --->
select which bus will be used (sdio bus) --->
Select which firmware will be used:.bin or firmware.h (Request .bin from system) --->
Driver Extern Function Select --->
Driver debug features --->
(mmc0) which mmc will be used (NEW)
[ ] Use GPIO interrupt (NEW)
[*] Use short GI support
(wlan%d) Setting wifi interface 1 name
[*] ENABLE scond interface
(p2p%d) Setting wifi interface 2 name
(pm_stayawake) Setting wifi pm stay awake modules name
(atbm_wlan) Setting wifi module driver name
(atbm_dev_wifi) Setting wifi platform device name
(0x007a) Setting wifi sdio vid (NEW)
(0x6051) Setting wifi sdio pid (NEW)
(ATBM606x_wifi_usb) set module name
(/system/etc/firmware/ATBM606x_fw_sdio.bin) set fw path name (NEW)
[*] altobeam wifi6 WiFi cfg80211 --->
[*] altobeam support wpa3 in new cfg80211 but less kernel version
[ ] altobeam support ble module --->
select which bus will be used
Use the arrow keys to navigate this window or press the hotkey of the item you wish to select followed by the <SPACE BAR>. Press <?> for additional information about this option.
(X) usb bus
( ) sdio bus
( ) spi bus
<Select> < Help >

如果选择 sdio 总线，需要选择是 sdio 中断还是 gpio 中断



a) 如果需要使用 GPIO 中断，

需要做如下修改，否则跳过此部分 ：

hal_apollo/Makefile 中，根据 platform 添加 GPIO 中断的宏。

```
ccflags-y += -DOPER_CLOCK_USE_SEM
ccflags-y += -DEXIT_MODULE_RESET_USB=0
ccflags-y += -DATBM_VIF_LIST_USE_RCU_LOCK
#ccflags-y += -DATBM_SUPPORT_SMARTCONFIG

ifeq ($(platform),PLATFORM_AMLOGIC_S805)
ccflags-y += -DCONFIG_ATBM_APOLLO_USE_GPIO_IRQ
endif
ifeq ($(platform),PLATFORM_FH_IPC)
ccflags-y += -DCONFIG_ATBM_APOLLO_USE_GPIO_IRQ
endif

ifeq ($(platform),PLATFORM_AMLOGIC)
#ccflags-y += -DCONFIG_ATBM_APOLLO_USE_GPIO_IRQ
endif
```

hal_apollo/apollo_plat.h 添加宏,该宏的值和顶层 Makefile 中 export 出来的值一样

```
*PLATFORM_FRIENDLY:based on linux3.086
*
*********************************************************
*/
#define PLATFORM_XUNWEI                         (1)
#define PLATFORM_SUN6I                          (2)
#define PLATFORM_FRIENDLY                       (3)
#define PLATFORM_SUN6I_64                       (4)
#define PLATFORM_CDLINUX                        (12)
#define PLATFORM_AMLOGIC_S805           (13)
#define PLATFORM_AMLOGIC_905            (8)
#define PLATFORM_FH_IPC                         (18)

#ifndef  ATBM_WIFI_PLATFORM
#define ATBM_WIFI_PLATFORM                      PLATFORM_XUNWEI
#endif
```

hal_apollo/atbm_platform.c 添加 GPIO 中断引脚号

AltoBeam Confidential Document

```
^M
#endif  //CONFIG_ATBM_APOLLO_USE_GPIO_IRQ^M
#endif^M
struct atbm_platform_data platform_data = {^M
#if (ATBM_WIFI_PLATFORM == 10)^M
        .mmc_id       = "mmc1",^M
#else^M
        .mmc_id       = "mmc2",^M
#endif^M
        .clk_ctrl     = NULL,^M
        .power_ctrl   = atbm_power_ctrl,^M
        .insert_ctrl  = atbm_insert_crtl,^M
#if(ATBM_WIFI_PLATFORM == PLATFORM_XUNWEI)^M
        .irq_gpio       = EXYNOS4_GPX2(4),^M
        .power_gpio     = EXYNOS4_GPC1(1),^M
#endif^M
#if(ATBM_WIFI_PLATFORM == PLATFORM_AMLOGIC_S805)^M
        .irq_gpio       = INT_GPIO_4,^M
        .power_gpio     = 0,^M
#endif^M
#if(ATBM_WIFI_PLATFORM == PLATFORM_AMLOGIC_905)^M
        .irq_gpio       = 100,^M
        .power_gpio     = 0,^M
#endif^M
#if(ATBM_WIFI_PLATFORM == PLATFORM_FRIENDLY)^M
        .power_gpio     = EXYNOS4_GPK3(2),^M
#endif^M
#if(ATBM_WIFI_PLATFORM == PLATFORM_FH_IPC)^M
        .irq_gpio       = 0,^M
#endif^M
        .reset_gpio = 0,^M
};^M
^M
struct atbm_platform_data *atbm_get_platform_data(void)^M
{^M
        return &platform_data;^M
```

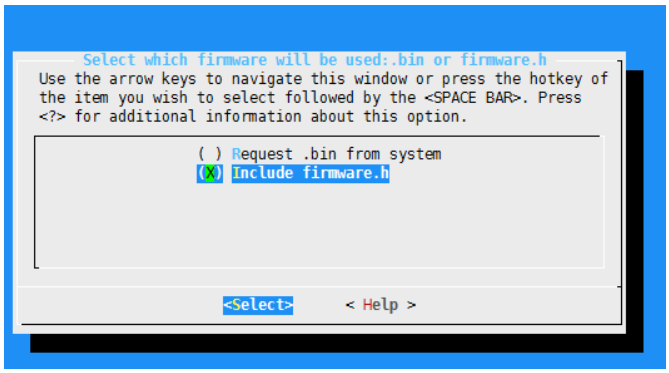
b) 整体设定完毕后，在驱动源码下执行 make 即可。

(2) 选择固件方式

可以选择独立于驱动之外的 bin 文件，或者选择包含在驱动里面的 hal_apollo/firmware_<bus>.h

```
--- Atbm Wireless Lan
[ ]   support wireless wext
      select which atbm Wi-Fi product  will be used: ATBM606x,default:ATBM606x (ATBM606x chip)  --->
      select which bus will be used (usb bus)  --->
      Select which firmware will be used:.bin or firmware.h (Request .bin from system)  --->
      Driver Extern Function Select  --->
      Driver debug features  --->
[*]   Use short GI support
(wlan%d) Setting wifi interface 1 name
[*]   ENABLE scond interface
(p2p%d) Setting wifi interface 2 name
(pm_stayawake) Setting wifi pm stay awake modules name
(atbm_wlan) Setting wifi module driver name
(atbm_dev_wifi) Setting wifi platform device name
(0x007a) Setting wifi usb vid
(0x6052) Setting wifi usb pid
(ATBM606x_wifi_usb) set module name
(/system/etc/firmware/ATBM606x_fw_usb.bin) set fw path name (NEW)
[*]   altobeam  wifi6 WiFi cfg80211  --->
[*]   altobeam support wpa3 in new cfg80211 but less kernel version
[ ]   altobeam support ble module  --->
```

AltoBeam Confidential Document



IPC 考虑到使用方便的原因一般选择 firmware.h，安卓平台一般都是用的 bin 文件。

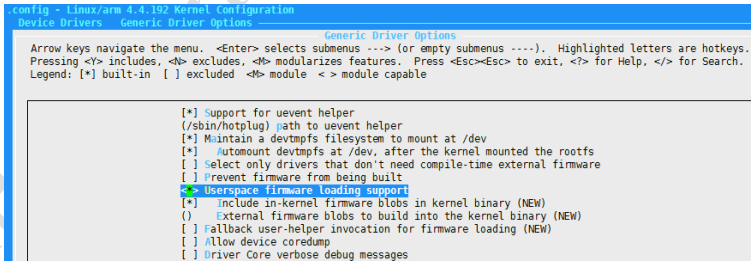
芯片型号	WiFi+BLE comb firmware
ATBM6062	firmware_usb_wifi_bt_comb_cronus.h
ATBM6062C	firmware_usb_wifi_bt_comb_clite.h
ATBM6162	firmware_usb_wifi_bt_comb_ocea.h
ATBM6132C	firmware_usb_wifi_bt_comb_ocea.h

2.2.2.1 使用 bin 文件

PS:

1）固件使用 bin 文件需要内核支持下载 firmware,所以在内核需要打开 FW_LOADER 宏,不打开没法正常下载固件。

```
Symbol: FW_LOADER [=y]
Type  : tristate
Prompt: Userspace firmware loading support
  Location:
    -> Device Drivers
(1)   -> Generic Driver Options
  Defined at drivers/base/Kconfig:80
  Selected by: IXP4XX_NPE [=n] && ARCH_IXP4XX [=n] || PCM
```



2）最终的固件只能放在内核预定义的路径

2.1）该路径定义于：kernel/drivers/base/firmware_class.c

AltoBeam Confidential Document

```
/* direct firmware loading support */
static char fw_path_para[256];
static const char * const fw_path[] = {
        fw_path_para,
        "/lib/firmware/updates/" UTS_RELEASE,
        "/lib/firmware/updates",
        "/lib/firmware/" UTS_RELEASE,
        "/lib/firmware"
};
```

2.2）在运行系统里面添加存放固件的路径

例如固件放的路径为/mnt/sdcard/firmware/添加路径的方法如下：

```
echo -n “/mnt/sdcard/firmware/” > /sys/module/firmware_class/parameters/path
```

2.3）ATBM WIFI6 Bin 固件名称

```
char firmware_bin[][32]={
#ifdef USB_BUS
        {"ATBM606x_fw_usb.bin"},
        {"ATBM606x_blecomb_usb.bin"}
#else
        {"ATBM606x_fw_sdio.bin"},
        {"ATBM606x_blecomb_sdio.bin"}
#endif
};
```

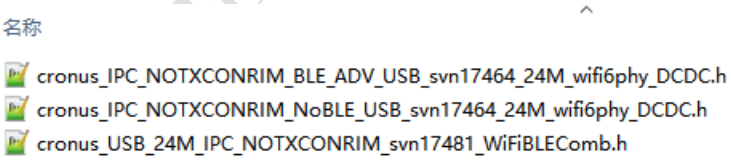
以 USB WIFI 为例：

ATBM606x_fw_usb.bin 为纯 WIFI 固件

ATBM606x_blecomb_usb.bin 为 WIFI+BLE 共存固件

2.2.2.2 使用.h 文件

再驱动根目录下的 firmware 目录放的是 wifi 的固件。



带 WiFiBLEComb 的名称的固件为 WIFI+BLE 连接共存的固件。

带 NoBLE 名称的固件为不支持 BLE 的固件。

将不支持 BLE 的固件改名为 firmware_<bus>.h 放到 hal_apollo 目录下。

例如：

1) 使用的是 usb 总线+不支持 BLE 的功能，需要放一个固件：

带 NoBLE 名称的固件改名为 fiwmare_usb.h 放到 hal_apollo/目录下

2) 使用的是 usb 总线+WIFI_BLE 共存的功能，需要放两个固件：

带 NoBLE 名称的固件改名为 firmware_usb.h 放到 hal_apollo/目录下

AltoBeam Inc. Confidential Document

带 WiFiBLEComb 的名称的固件改名为 firmware_usb_wifi_bt_comb.h 放到 hal_apollo/目录下

(3) 驱动一些扩展功能

```
--- Atbm Wireless Lan
[ ]   support wireless wext
      select which atbm Wi-Fi product  will be used: ATBM606x,default:ATBM606x (ATBM606x chip)
      select which bus will be used (usb bus)  --->
      Select which firmware will be used:.bin or firmware.h (Include firmware.h)  --->
      Driver Extern Function Select  --->
      Driver debug features  --->
(CN)  which country set :US/CN/JP/00
[*]   Use short GI support
(wlan%d) Setting wifi interface 1 name
[*]   ENABLE scond interface
(p2p%d) Setting wifi interface 2 name
(pm_stayawake) Setting wifi pm stay awake modules name
(atbm_wlan) Setting wifi module driver name
(atbm_dev_wifi) Setting wifi platform device name
(0x007a) Setting wifi usb vid
(0x6052) Setting wifi usb pid
(ATBM606x_wifi_usb) set module name
[*]   altobeam  wifi6 WiFi cfg80211  --->
[ ]   altobeam support wpa3 in new cfg80211 but less kernel version
[ ]   altobeam support ble module  --->
[ ]   altobeam support ble coexist  --->
```

```
[*] Enable wifi interface bridge function
[*] Enable Tx no confirm function to enhance performance
[ ] Enable early suspend function for some platform power save
[*] Enable rx monitor function to receive all package
[ ]   Enable rx monitor header prism
[ ] Enable skb debug function to debug skb alloc and free
[ ] Enable memory debug function to debug memory alloc and free
[ ] Enable 2.4g useing 5g channel function ,only support special frequnce
[*] Enable only support HT20
[ ] Enabel usb aggr tx funciton to enchance tx performance
[ ] Enable usb use dam buff for xmit
[*] The hardware may not generate an interrupt when the urb ends
[ ] Enable usb data send directly function
[ ] Enable usb wakeup reload fw function
[ ] Enable hw do tcp/ip checksum function
[*] Enable P2P
[ ] enable sw enc function
[*] enabel dev_ctrl api
[*] enable modules fs
[ ] Enable loader driver fast function
[*] Enable iwpriv some prive func
[*] Enable WPA3 support,but make sure the kernel support sae before enabled
[*] Enable driver set up tx/rx BA
```

a) Enable wifi interface bridge function
选择驱动是否支持桥接

b) Enable Tx no confirm function to enhance performance
这个功能默认是打开的。

c) Enable early suspend function for some platform power save
与平台相关，安卓系统层支持休眠时候需要打开。

d) Enable rx monitor function to receive all package
驱动是否支持进入监听状态的功能，默认打开
子选项：Enable rx monitor header prism
Monitor 头部修改为 PRISM 格式，默认为 RATIO 格式

e) Enable skb debug function to debug skb alloc and free
打开 skb 泄露 debug 的功能，此功能通常不打开

f) Enable memory debug function to debug memory alloc and free
打开 memory 泄露 debug 的功能，此功能通常不打开

g) Enable 2.4g useing 5g channel function ,only support special frequnce
是否使用 5G 信道作为特殊频点，默认关闭状态

h) Enabel usb aggr tx funciton to enchance tx performance
打开 usb 聚合发送数据包的功能，目前关闭，cpu 频率较低时可以打开

i) Enable usb use dam buff for xmit
使用 usb 的 dma buff,一般打开 usb 聚合时打开此功能

j) Enable usb data send directly function
cpu 频率较低时打开此功能

k) The hardware may not generate an interrupt when the urb ends
该配置打开会减少 usb 中断，需要 usb host 支持，如果支持不好会导致系统卡死。
目前已知的君正&多方平台需要将该配置关闭。

l) Enable usb wakeup reload fw function
android 平台休眠唤醒时是否重新 load 固件，默认关闭

m) Enable hw do tcp/ip checksum function
是否使能 aresB 的 check sum 功能，默认关闭

n) Enable P2P
使能 P2P mirecast 功能

o) enable sw enc function
支持软解密功能，wep 加密需要开启，会导致 cpu 使用率增加

p) enabel dev_ctrl api
使能 dev ioctl 相关命令功能

q) enable modules fs

AltoBeam Inc. Confidential Document

使能 modules fs 功能，打开此功能会在/sys/module/<driver_name>/目录下生成 atbm_fs 目录

该目录可以进行功能调试

r) Enable loader driver fast function

打开此功能使用并发的方式缩短加载 usb 驱动的时间，但有一些平台使用该方式会导致 usb 通信出错

s) Enable iwpriv some prive func

打开此功能支持往 beacon/probe 帧插入私有数据的命令

t) Enable driver set up tx/rx BA

使能 tx/rx ampdu 聚合功能

(4) 内部调试使用

```
--- Atbm Wireless Lan
[ ]   support wireless wext
      select which atbm Wi-Fi product  will be used:ATBM602x,ATBM603x,ATBM604x,de
      select which bus will be used (usb bus)  --->
      Select which firmware will be used:.bin or firmware.h (Request .bin from sy
      Driver Extern Function Select  --->
      Driver debug features  --->
[*]   Use short GI support
(wlan%d) Setting wifi interface 1 name
[*]   ENABLE scond interface
(p2p%d) Setting wifi interface 2 name
(pm_stayawake) Setting wifi pm stay awake modules name
(atbm_wlan) Setting wifi module driver name
(atbm_dev_wifi) Setting wifi platform device name
(+)
```

(5) 设置默认国家码

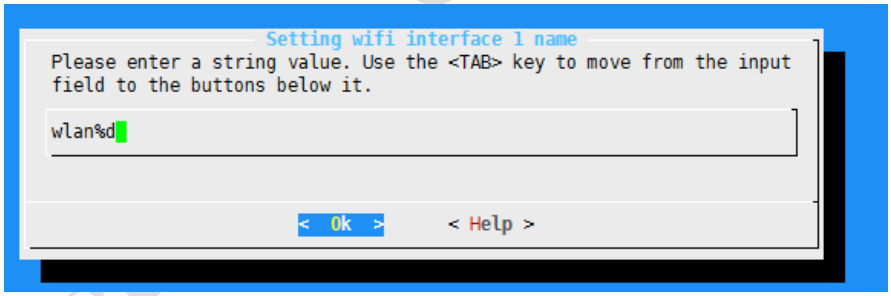
```
--- Atbm Wireless Lan
[ ]   support wireless wext
      select which atbm Wi-Fi product  will be used: ATBM606x,default:ATBM606x (ATBM606x chip)
      select which bus will be used (usb bus)  --->
      Select which firmware will be used:.bin or firmware.h (Include firmware.h)  --->
      Driver Extern Function Select  --->
      Driver debug features  --->
(CN)  which country set :US/CN/JP/00
[*]   Use short GI support
(wlan%d) Setting wifi interface 1 name
[*]   ENABLE scond interface
(p2p%d) Setting wifi interface 2 name
(pm_stayawake) Setting wifi pm stay awake modules name
(atbm_wlan) Setting wifi module driver name
(atbm_dev_wifi) Setting wifi platform device name
(0x007a) Setting wifi usb vid
(0x6052) Setting wifi usb pid
(ATBM606x_wifi_usb) set module name
[*]   altobeam  wifi6 WiFi cfg80211  --->
[ ]   altobeam support wpa3 in new cfg80211 but less kernel version
[ ]   altobeam support ble module  --->
[ ]   altobeam support ble coexist  --->
```

默认可以配置支持的国家码：

国家码	支持的信道
US	11
CN	13
JP	14
00	14

(6) 修改 wifi 接口名称

```
--- Atbm Wireless Lan
[ ]   support wireless wext
      select which atbm Wi-Fi product  will be used: ATBM606x,default:ATBM606x (ATBM606x chip)  --->
      select which bus will be used (usb bus)  --->
      Select which firmware will be used:.bin or firmware.h (Include firmware.h)  --->
      Driver Extern Function Select  --->
      Driver debug features  --->
[*]   Use short GI support
(wlan%d) Setting wifi interface 1 name
[*]   ENABLE scond interface
(p2p%d) Setting wifi interface 2 name
(pm_stayawake) Setting wifi pm stay awake modules name
(atbm_wlan) Setting wifi module driver name
(atbm_dev_wifi) Setting wifi platform device name
(0x007a) Setting wifi usb vid
(0x6052) Setting wifi usb pid
(ATBM606x_wifi_usb) set module name
[*]   altobeam  wifi6 WiFi cfg80211  --->
[*]   altobeam support wpa3 in new cfg80211 but less kernel version
[ ]   altobeam support ble module  --->
```



wifi 接口默认的名称是：wlan0 以及 p2p0.

客户可以根据需要执行修改：

1）客户想使用 wlan0 以及 wlan1，

可直接修改 p2p%d 修改为 wlan%d 即可。

2）客户想使用 wlan 以及 p2p 接口

将 wlan%d 修改为 wlan

将 p2p%d 修改为 p2p

注意：

去掉 ENABLE scond interface 配置项，则只要单独的 wlan%d 网口，而没有两个网口。

（7） 修改驱动名称以及挂载结点名称

```
--- Atbm Wireless Lan
[ ]   support wireless wext
      select which atbm Wi-Fi product  will be used: ATBM606x,default:ATBM606x (ATBM606x chip)  --->
      select which bus will be used (usb bus)  --->
      Select which firmware will be used:.bin or firmware.h (Include firmware.h)  --->
      Driver Extern Function Select  --->
      Driver debug features  --->
[*]   Use short GI support
(wlan%d) Setting wifi interface 1 name
[*]   ENABLE scond interface
(p2p%d) Setting wifi interface 2 name
(pm_stayawake) Setting wifi pm stay awake modules name
(atbm_wlan) Setting wifi module driver name
(atbm_dev_wifi) Setting wifi platform device name
(0x007a) Setting wifi usb vid
(0x6052) Setting wifi usb pid
(ATBM606x_wifi_usb) set module name
[*]   altobeam  wifi6 WiFi cfg80211  --->
[*]   altobeam support wpa3 in new cfg80211 but less kernel version
[ ]   altobeam support ble module  --->
```

根据客户需要执行修改，编译出来生成的驱动名称。
以及在系统中挂载驱动 lsmod 看到的驱动名称。

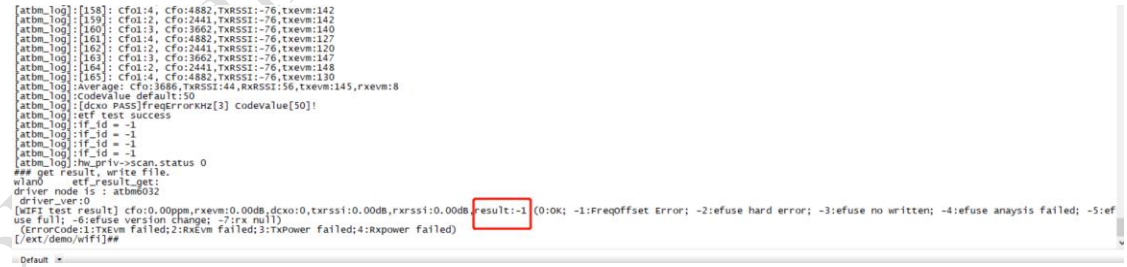
2.2.7.1 注意【重要】

需要注意驱动名称一定需要以如下格式开头：

atbm606x 芯片为：atbm606x_xxxxx

该命名会影响到 Golden 产测结果的计算。

出问题情况如下：



AltoBeam Inc. Confidential Document

(8) 配置驱动自带的 CFG80211 功能

```
-*- Atbm Wireless Lan
[ ]   support wireless wext
      select which atbm Wi-Fi product  will be used: ATBM606x,default:ATBM606x (ATBM606x chip)  --->
      select which bus will be used (usb bus)  --->
      Select which firmware will be used:.bin or firmware.h (Include firmware.h)  --->
      Driver Extern Function Select  --->
      Driver debug features  --->
[*]   Use short GI support
(wlan%d) Setting wifi interface 1 name
[*]   ENABLE scond interface
(p2p%d) Setting wifi interface 2 name
(pm_stayawake) Setting wifi pm stay awake modules name
(atbm_wlan) Setting wifi module driver name
(atbm_dev_wifi) Setting wifi platform device name
(0x007a) Setting wifi usb vid
(0x6052) Setting wifi usb pid
(ATBM606x_wifi_usb) set module name
[*]   altobeam  wifi6 WiFi cfg80211  --->
[*]   altobeam support wpa3 in new cfg80211 but less kernel version
[ ]   altobeam support ble module  --->
```

```
-*- altobeam  wifi6 WiFi cfg80211
[*]   altobeam wifi6 WiFi cfg80211 wext
[*]   altobeam wifi6 WiFi cfg80211 CRDA
[*]     altobeam wifi cfg80211 support country code
```

WIFI6 需要依赖的内核版本比较高，所以为了适应低版本的内核，WIFI 驱动自带 CFG80211。

驱动目录下的 wireless 是 linux 内核中的 cfg80211 模块。

将 Wireless 放在这主要是 wifi6 功能需要在 linux 版本 5.7 之后才能支持的比较好。

而 5.7 之前的版本要支持 wifi6 需要客户更新 cfg80211，所以建议直接使用我们提供的 cfg80211，而 5.7 以后的版本可能会需要一些更新，建议直接使用内核自带的 cfg80211。

目前 Wireless 只在全志板子的内核 3.15，4.9.84 上测试正常，其他版本的内核可能会有兼容问题。

(9) 自定义 ATBM WIFI 驱动支持 WPA3

```
--- Atbm Wireless Lan
[ ]   support wireless wext
      select which atbm Wi-Fi product  will be used: ATBM606x,default:ATBM606x (ATBM606x chip)  --->
      select which bus will be used (usb bus)  --->
      Select which firmware will be used:.bin or firmware.h (Request .bin from system)  --->
      Driver Extern Function Select  --->
      Driver debug features  --->
[*]   Use short GI support
(wlan%d) Setting wifi interface 1 name
[*]   ENABLE scond interface
(p2p%d) Setting wifi interface 2 name
(pm_stayawake) Setting wifi pm stay awake modules name
(atbm_wlan) Setting wifi module driver name
(atbm_dev_wifi) Setting wifi platform device name
(0x007a) Setting wifi usb vid
(0x6052) Setting wifi usb pid
(ATBM606x_wifi_usb) set module name
(/system/etc/firmware/ATBM606x_fw_usb.bin) set fw path name (NEW)
[*]   altobeam  wifi6 WiFi cfg80211  --->
[*]   altobeam support wpa3 in new cfg80211 but less kernel version
[ ]   altobeam support ble module  --->
```

该配置项是如果有客户小于 Linux 3.8.0 内核版本的 cfg80211 上移植了 WPA3 的功能，那么 ATBM WIFI 驱动就无法根据内核的版本准备的判断内核是否支持 WPA3 功能，此时需要将该配置打开，

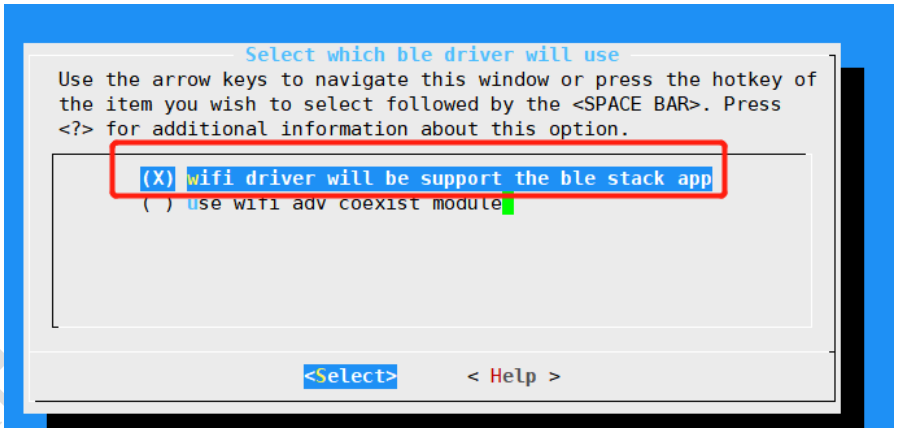
ATBM WIFI 驱动就会认为内核是支持 WPA3 功能的。

(10) BLE 支持

```
--- Atbm Wireless Lan
[ ]   support wireless wext
      select which atbm Wi-Fi product  will be used: ATBM606x,default:ATBM606x (ATBM606x chip)  --->
      select which bus will be used (usb bus)  --->
      Select which firmware will be used:.bin or firmware.h (Request .bin from system)  --->
      Driver Extern Function Select  --->
      Driver debug features  --->
[*]   Use short GI support
(wlan%d) Setting wifi interface 1 name
[*]   ENABLE scond interface
(p2p%d) Setting wifi interface 2 name
(pm_stayawake) Setting wifi pm stay awake modules name
(atbm_wlan) Setting wifi module driver name
(atbm_dev_wifi) Setting wifi platform device name
(0x007a) Setting wifi usb vid
(0x6052) Setting wifi usb pid
(ATBM606x_wifi_usb) set module name
(/system/etc/firmware/ATBM606x_fw_usb.bin) set fw path name
[*]   altobeam  wifi6 WiFi cfg80211  --->
[*]   altobeam support wpa3 in new cfg80211 but less kernel version
[ ]   altobeam support ble module  --->
```

```
--- altobeam support ble module
      Select which ble driver will use (wifi driver will be support the ble stack app)  --->
[*]   Wifi BLE comb Firmware for certification purposes
[ ]   Only wifi_ble comb firmware,There is no pure WiFi firmware
```

2.2.10.1 WIFI + BLE 连接共存配置



需要将带有 WiFiBLEComb 名称的固件改名为 fiwmare_<bus>_wifi_bt_comb.h 放到 hal_apollo 目录下。

```
[yuzhihuang@ubuntu_sz_2nd firmware]$ ls -lh
total 2.5M
-rwxr--r-- 1 yuzhihuang software 971K  5月 26 10:39 cronus_NOTXCONFIRM_USB_svn17117_24M_wifi6phy_DCDC.h
-rwxr--r-- 1 yuzhihuang software 1.6M  5月 26 10:39 cronus_USB_24M_IPC_NOTXCONRIM_svn17192_WiFiBLEComb.h
[yuzhihuang@ubuntu_sz_2nd firmware]$
```

例如：使用的是 usb 总线，则名称为:fiwmare_usb_wifi_bt_comb.h

2.2.10.2 增加一个 WIFI+BLE 过认证的固件

```
--- altobeam support ble module
      Select which ble driver will use (wifi driver will be support the ble stack app)  --->
[ ]   Wifi BLE comb Firmware for certification purposes
[ ]   Only wifi_ble comb firmware,There is no pure WiFi firmware
```

驱动包含一个专门过认证的固件，可以通过 iwpriv wlan0 common fw_reset,2 进行切换。

需要将带有 WiFiBLEComb CERT 名称的固件改名为 fiwmare_<bus>_wifi_bt_comb_cert.h 放到 hal_apollo 目录下。

```
[yuzhihuang@ubuntu_sz_2nd firmware]$ ls -lh
total 2.5M
-rwxr--r-- 1 yuzhihuang software 971K  5月 26 10:39 cronus_NOTXCONFIRM_USB_svn17117_24M_wifi6phy_DCDC_h
-rwxr--r-- 1 yuzhihuang software 1.6M  5月 26 10:39 cronus_USB_24M_IPC_NOTXCONRIM_svn17192_WiFiBLEComb.h
[yuzhihuang@ubuntu_sz_2nd firmware]$
```

例如：使用的是 usb 总线，则名称为:firmware_usb_wifi_bt_comb_cert.h

2.2.10.3 去掉纯 WIFI 的固件

```
--- altobeam support ble module
      Select which ble driver will use (wifi driver will be support the ble stack app)  --->
[*]   Wifi BLE comb Firmware for certification purposes
[ ]   Only wifi_ble comb firmware,There is no pure WiFi firmware
```

打开该配置会拿掉纯 WIFI 的固件，只包含 WIFI+BLE 共存的固件，节省 160k 左右的 flash 内存。

2.3 SDIO WIFI 移植配置说明

sdio 通信有 sdio 中断和 GPIO 中断通信方式。

(1) 注意

这里需要注意下如果 mmc host 进行了如下配置了：

AltoBeam Inc. Confidential Document

```
void mmc_rescan(struct work_struct *work)
{
    struct mmc_host *host =
        container_of(work, struct mmc_host, detect.work);
    int i;

    if (host->trigger_card_event && host->ops->card_event) {
        host->ops->card_event(host);
        host->trigger_card_event = false;
    }

    if (host->rescan_disable)
        return;

    /* If there is a non-removable card registered, only scan once */
    if ((host->caps & MMC_CAP_NONREMOVABLE) && host->rescan_entered)
        return;
    host->rescan_entered = 1;

    mmc_bus_get(host);
```

代表 mmc host 不允许 sdio 从设备进行热插拔，调用 mmc rescan 直接返回,不会去扫卡。所以此时驱动直接加载卸载即可。

有一些平台刚上电 sdio 不稳定就需要复位一下 sdio wifi，那么就需要增加复位扫卡的动作。

(2) SDIO 中断方式

说明：

以君正 T31 平台为例。

2.3.2.1 修改驱动根目录的 Makefile

自定义一个 platform

```
#PLATFORM_ROCKCHIP_3229             15
#PLATFORM_ROCKCHIP_3229_ANDROID8    16
#PLATFORM_HS_IPC                    17
#PLATFORM_SIGMASTAR                 18
#PLATFORM_HI3516EV200               19
#PLATFORM_XUNWEI_2G                 20
#PLATFORM_NVT98517                  21
#PLATFORM_INGENICT31                22
#PLATFORM_INGENICT41                23


platform ?= PLATFORM_INGENICT31
#ATBM_BUILD_IN_KERNEL?=
```

增加一个编译配置项，增加内核路径和工具链路径用于单独编译驱动使用

需要注意 ATBM_WIFI_PLATFORM 值为 24，在 Makefile 底下添加一个

Confidential Document

2.3.2.2 修改 hal_apollo/apollo_plat.h

增加君正 T31 平台定义 ATBM_WIFI_PLATFORM 宏值为 24

```
*/
#define PLATFORM_XUNWEI                     1
#define PLATFORM_SUN6I                      2
#define PLATFORM_FRIENDLY                   3
#define PLATFORM_SUN6I_64                   4
#define PLATFORM_HI3798M                    5
#define PLATFORM_HI3518E                    6
#define PLATFORM_X86PC                      7
#define PLATFORM_AMLOGIC                    8
//#define PLATFORM_AMLOGIC_905              8
#define PLATFORM_AMLOGIC_905X               9
#define PLATFORM_ROCKCHIP                   10
#define PLATFORM_MSTAR                      11
#define PLATFORM_CDLINUX                    12
#define PLATFORM_AMLOGIC_S805               13
#define PLATFORM_HIS_LINUX_3_4              14
#define PLATFORM_ROCKCHIP_3229              15
#define PLATFORM_ROCKCHIP_3229_ANDROID8     16
#define PLATFORM_HS_IPC                     17
#define PLATFORM_SIGMASTAR                  18
#define PLATFORM_HI3516EV200                19
#define PLATFORM_XUNWEI_2G                  20
#define PLATFORM_NVT98517                   21
#define PLATFORM_AK_IPC                     22
#define PLATFORM_SUN8I                      23
#define PLATFORM_INGENICT31                 24
#define PLATFORM_INGENICT41                 25
#define PLATFORM_SUN50IWP1                  26
#define PLATFORM_KALI                       27
#define PLATFORM_XM530                      28
#define PLATFORM_GK7202V330                 29
#define PLATFORM_INGENIC_X2000              30

#ifndef  ATBM_WIFI_PLATFORM
#define ATBM_WIFI_PLATFORM          PLATFORM_INGENICT31
#endif

#define APOLLO_1505  0
```

为了保险这里增加了 ATBM_WIFI_PLATFORM 的宏定义，如果定义了这里那么 1.3.2.3 小节可以忽略。

AltoBeam Inc. Document

```
#define PLATFORM_AK_IPC                 22
#define PLATFORM_SUN8I                  23
#define PLATFORM_INGENICT31             24
#define PLATFORM_INGENICT41             25
#define PLATFORM_SUN50IWP1              26
#define PLATFORM_KALI                   27
#define PLATFORM_XM530                  28
#define PLATFORM_GK7202V330             29
#define PLATFORM_INGENIC_X2000          30

#ifndef  ATBM_WIFI_PLATFORM
#define ATBM_WIFI_PLATFORM              PLATFORM_INGENICT31
#endif

#define APOLLO_1505  0
#define APOLLO_1601  1
#define APOLLO_1606  0
#define APOLLO_C     2
#define APOLLO_D     3
#define APOLLO_E     4
#define APOLLO_F     5
#define ATHENA_B     6
```

2.3.2.3 定义 ATBM_WIFI_PLATFORM 的值用于全局

修改 hal_apollo/Makefile

```
#ccflags-y += -DCONFIG_USB_DATA_XMIT_DIRECTLY
#ccflags-y += -DCONFIG_USE_DMA_ADDR_BUFFER
#ccflags-y += -DCONFIG_USB_AGGR_URB_TX
#ccflags-y += -DCONFIG_TX_NO_CONFIRM
#ccflags-y += -DATBM_IPC_ASSOC
#ccflags-y += -DATBM_PRIVATE_IE
#ccflags-y += -DSIGMASTAR_FILTER_MACADDR_ONLY
#ccflags-y += -DCONFIG_ATBM_5G_PRETEND_2G
endif
ifeq ($(TX_NO_CONFIRM),y)
#ccflags-y += -DCONFIG_USE_DMA_ADDR_BUFFER
ccflags-y += -DCONFIG_USB_AGGR_URB_TX
ccflags-y += -DCONFIG_TX_NO_CONFIRM
endif
ifeq ($(SDIO_BUS),y)
ccflags-y += -DATBM_WIFI_PLATFORM=24
ccflags-y += -DMCAST_FWDING
ccflags-y += -DRESET_CHANGE
ccflags-y += -DSDIO_BUS
#ccflags-y += -DCONFIG_SDIO_IRQ_THREAD_PROCESS_DATA
ccflags-y += -DCONFIG_ATBM_SDIO_TX_HOLD
ccflags-y += -DCONFIG_ATBM_SDIO_TX_THREAD_FIFO
#ccflags-y += -DCONFIG_SDIO_TXRX_WORKQUEUE
#ccflags-y += -DATBM_SDIO_TXRX_ENHANCE
#ccflags-y += -DATBM_WSM_SDIO_TX_MULT
#ccflags-y += -DRESET_CHIP
endif
ifeq ($(SPI_BUS),y)
ccflags-y += -DSPI_BUS
endif
```

AltoBeam Confidential Document

2.3.2.4 修改 hal_apollo/atbm_platform.c

如果需要增加复位扫卡动作见后面【2.3.4 复位&扫卡动作】

(3) GPIO 中断方式

说明：

以 amlogic s905 平台为例

2.3.3.1 修改驱动根目录的 Makefile

自定义一个 platform

```
#PLATFORM_HIS_LINUX_3_4              14
#PLATFORM_ROCKCHIP_3229              15
#PLATFORM_ROCKCHIP_3229_ANDROID8     16
#PLATFORM_HS_IPC                     17
#PLATFORM_SIGMASTAR                  18
#PLATFORM_HI3516EV200                19
#PLATFORM_XUNWEI_2G                  20
#PLATFORM_NVT98517                   21
#PLATFORM_INGENICT31                 22
#PLATFORM_INGENICT41                 23


platform ?= PLATFORM_AMLOGIC_905
#ATBM_BUILD_IN_KERNEL?=

```

增加一个编译配置项，增加内核路径和工具链路径用于单独编译驱动使用

需要注意 ATBM_WIFI_PLATFORM 值为 8

```
ifeq ($(platform),PLATFORM_AMLOGIC_905)
ifeq ($(sys),Android)
#KERDIR:=/wifi_prj/staff/zhouzhanchao/amlogic/release_s905_l/out/target/product/p200/obj/KERNEL_OBJ/
KERDIR:=/wifi_prj/staff/mengxuehong/s905l/S905L/out/target/product/p201_iptv/obj/KERNEL_OBJ/
CROSS_COMPILE:=/ssd-home/mengxuehong/buildTool1/gcc-linaro-aarch64-linux-gnu-4.9-2014.09_linux/bin/aa
else
#KERDIR:=/wifi_prj/staff/panxuqiang/64bi_driver/cqa64_linux_qt5.3.2/lichee/linux-3.10/
endif
export
ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=8
arch:=arm64
endif
```

在最后的增加一个配置项用于在内核目录直接编译 modules 时候的配置

需要主要 ATBM_WIFI_PLATFORM 值为 8

```
ifeq ($(platform),PLATFORM_AMLOGIC_905)
export
ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=8
endif
```

2.3.3.2 打开支持 GPIO 中断配置

```
        select which atbm Wi-Fi product  will be used:ATBM602x,ATBM603x,ATBM604x,de
        select which bus will be used (sdio bus)  --->
        Select which firmware will be used:.bin or firmware.h (Include firmware.h)
        Driver Extern Function Select  --->
        Driver debug features  --->
(mmc0) which mmc will be used
[ ]   Use GPIO interrupt
[*]   Use short GI support
(wlan%d) Setting wifi interface 1 name
[*]   ENABLE scond interface
(p2p%d) Setting wifi interface 2 name
(pm_stayawake) Setting wifi pm stay awake modules name
(atbm_wlan) Setting wifi module driver name
(atbm_dev_wifi) Setting wifi platform device name
└(+)
```

2.3.3.3 修改使用平台的 mmc 口

根据实际使用的 mmc 口进行配置。

一般平台有两个 mmc 口，mmc0 或者是 mmc1。

```
        select which atbm Wi-Fi product  will be used:ATBM602x,ATBM603x,ATBM604x,de
        select which bus will be used (sdio bus)  --->
        Select which firmware will be used:.bin or firmware.h (Include firmware.h)
        Driver Extern Function Select  --->
        Driver debug features  --->
(mmc0) which mmc will be used
[ ]   Use GPIO interrupt
[*]   Use short GI support
(wlan%d) Setting wifi interface 1 name
[*]   ENABLE scond interface
(p2p%d) Setting wifi interface 2 name
(pm_stayawake) Setting wifi pm stay awake modules name
(atbm_wlan) Setting wifi module driver name
(atbm_dev_wifi) Setting wifi platform device name
└(+)
```

2.3.3.4 修改 hal_apollo/apollo_plat.h

增加一个平台宏定义值为 8

AltoBeam Confidential Document

```
#define PLATFORM_XUNWEI             (1)
#define PLATFORM_SUN6I              (2)
#define PLATFORM_FRIENDLY           (3)
#define PLATFORM_SUN6I_64           (4)
#define PLATFORM_CDLINUX            (12)
#define PLATFORM_AMLOGIC_S805       (13)
#define PLATFORM_AMLOGIC_905        (8)
#define PLATFORM_ANYKA_SDIO         (22)
#define PLATFORM_INGENICT31         (23)


#ifndef  ATBM_WIFI_PLATFORM
#define ATBM_WIFI_PLATFORM          PLATFORM_AMLOGIC_905
#endif
```

2.3.3.5 修改 hal_apollo/atbm_platform.c

增加一个编译时候打印的信息

```
#if (ATBM_WIFI_PLATFORM == PLATFORM_AMLOGIC_905)
 #define PLATFORMINF      "amlogic_905"
 #endif
```

如果有不同版本的内核并且有较差异需要增加进来

```
#if ((ATBM_WIFI_PLATFORM == PLATFORM_AMLOGIC_S805) || (ATBM_WIFI_PLATFORM == PLATFORM_AMLOGIC_905))

#if (LINUX_VERSION_CODE < KERNEL_VERSION(3, 14, 0))
extern void wifi_teardown_dt(void);
extern int wifi_setup_dt(void);
#endif
#endif //#if (ATBM_WIFI_PLATFORM == PLATFORM_AMLOGIC_S805)
```

中断相关的声明定义

```
#if(ATBM_WIFI_PLATFORM == PLATFORM_AMLOGIC_905)

#if (LINUX_VERSION_CODE >= KERNEL_VERSION(3, 14, 0))
 extern int wifi_irq_num(void);
 #endif

u32 atbm_wlan_get_oob_irq(void)
 {
     u32 host_oob_irq = 0;

#if (LINUX_VERSION_CODE < KERNEL_VERSION(3, 14, 0))
     host_oob_irq = INT_GPIO_4;
#else
     host_oob_irq = wifi_irq_num();
#endif
     atbm_printk_platform("host_oob_irq: %d \r\n", host_oob_irq);

     return host_oob_irq;
 }
 #endif
```

struct atbm_platform_data platform_data 结构中进行中断号初始化

```
#if(ATBM_WIFI_PLATFORM == PLATFORM_AMLOGIC_905)
    .irq_gpio    = 100,
    .power_gpio = 0,
#endif
```

在 atbm_plat_request_gpio_irq 函数中增加 GPIO 中断的初始化

```
#if(ATBM_WIFI_PLATFORM == PLATFORM_AMLOGIC_905)
    bgf_irq = atbm_wlan_get_oob_irq();
    atbm_printk_platform("atbm_plat_request_gpio_irq \n");
    /* Request the IRQ */
    ret =  request_threaded_irq(bgf_irq, atbm_gpio_hardirq,
                atbm_gpio_irq,
                IORESOURCE_IRQ | IORESOURCE_IRQ_HIGHEDGE | IORESOURCE_IRQ_SHAREABLE,
                "atbm_wlan_irq", self);

    if (WARN_ON(ret))
        goto ↓err;
```

在 atbm_plat_free_gpio_irq 函数增加 GPIO 中断的反初始化

```
#if(ATBM_WIFI_PLATFORM == PLATFORM_AMLOGIC_905)
    disable_irq(atbm_bgf_irq);
    free_irq(atbm_bgf_irq,self);
#elif (ATBM_WIFI_PLATFORM == PLATFORM_AMLOGIC_S805)
    //do nothing
    disable_irq(atbm_bgf_irq);
    free_irq(atbm_bgf_irq,self);
#else
    disable_irq_wake(atbm_bgf_irq);
    free_irq(atbm_bgf_irq,self);
    gpio_free(pdata->irq_gpio);
#endif
```

(4) 复位&扫卡动作

2.3.4.1 注册

这里主要是在 hal_apollo/atbm_platform.c 里面的 struct atbm_platform_data 结构体的两个函数实现：

```
struct atbm_platform_data platform_data = {
#ifdef SDIO_BUS
    .mmc_id         = CONFIG_ATBM_SDIO_MMC_ID,
    .clk_ctrl       = NULL,
    .power_ctrl     = atbm_power_ctrl,
    .insert_ctrl    = atbm_insert_crtl,
#if(ATBM_WIFI_PLATFORM == PLATFORM_XUNWEI)
    .irq_gpio    = EXYNOS4_GPX2(4),
    .power_gpio = EXYNOS4_GPC1(1),
#endif
#if(ATBM_WIFI_PLATFORM == PLATFORM_AMLOGIC_S805)

    .irq_gpio    = INT_GPIO_4,
    .power_gpio = 0,
#endif
#if(ATBM_WIFI_PLATFORM == PLATFORM_AMLOGIC_905)
    .irq_gpio    = 100,
    .power_gpio = 0,
#endif
#if(ATBM_WIFI_PLATFORM == PLATFORM_FRIENDLY)
```

增加平台的扫卡函数以及控制 wifi 复位的 GPIO 号。

```
#if (ATBM_WIFI_PLATFORM == PLATFORM_INGENICT31)
#define PLATFORMINF "ingenict31"
extern int jzmmc_manual_detect(int index, int on);
static int WL_REG_EN = 32+25;
#endif
```

2.3.4.2 复位

在 hal_apollo/atbm_platform.c 里面 atbm_power_ctrl --> atbm_platform_power_ctrl 函数中增加对 wifi 的复位操作

```
#if (ATBM_WIFI_PLATFORM == PLATFORM_INGENICT31)
{
    if(enabled){
        atbm_printk_platform("[%s] reset altobeam wifi !\n",__func__);

        gpio_request(WL_REG_EN, "sdio_wifi_power_on");

        atbm_printk_platform("PLATFORM_INGENICT31 SDIO WIFI_RESET 0 \n");
        gpio_direction_output(WL_REG_EN, 0);
        msleep(300);
        atbm_printk_platform("PLATFORM_INGENICT31 SDIO WIFI_RESET 1 \n");
        gpio_direction_output(WL_REG_EN, 1);
        msleep(100);
    }
}
#endif//(ATBM_WIFI_PLATFORM == PLATFORM_INGENICT31)
```

2.3.4.3 扫卡

1) 如果平台有提供扫卡函数那么直接调用即可

在 hal_apollo/atbm_platform.c 里面 atbm_insert_crtl --> atbm_platform_insert_crtl 函数增加扫卡的动作

```
#if (ATBM_WIFI_PLATFORM == PLATFORM_INGENICT31)
{

    mdelay(100);
    jzmmc_manual_detect(1, enabled);
    atbm_printk_platform("============platform insert crtl====== enable=%d\n", enabled);

}
```

2) 如果平台没有提供对外的 mmc rescan 函数那么需要自己实现一个

在 hal_apollo/apollo_sdio.c 的 atbm_sdio_init 函数里面已经实现

```
#if ((ATBM_WIFI_PLATFORM != 10) && (ATBM_WIFI_PLATFORM != PLATFORM_AMLOGIC_S805)\
    && (ATBM_WIFI_PLATFORM != PLATFORM_AMLOGIC_905))

    ret = atbm_sdio_on(pdata);
    if (ret)
        goto err_on;
#endif
    atbm_wtd_init();
    return 0;

#if ((ATBM_WIFI_PLATFORM != 10) && (ATBM_WIFI_PLATFORM != PLATFORM_AMLOGIC_S805)\
    && (ATBM_WIFI_PLATFORM != PLATFORM_AMLOGIC_905))

err_on:
    if (pdata->power_ctrl)
        pdata->power_ctrl(pdata, false);
#endif
err_power:
    if (pdata->clk_ctrl)
        pdata->clk_ctrl(pdata, false);
err_clk:
    sdio_unregister_driver(&sdio_driver);
err_reg:
    return ret;
```

```
#if ((ATBM_WIFI_PLATFORM != 10) && (ATBM_WIFI_PLATFORM != PLATFORM_AMLOGIC_S805) \
    && (ATBM_WIFI_PLATFORM != PLATFORM_AMLOGIC_905))

static int atbm_sdio_on(const struct atbm_platform_data *pdata)
{
    int ret = 0;
    if (pdata->insert_ctrl)
        ret = pdata->insert_ctrl(pdata, true);
    msleep(200);
    atbm_detect_card(pdata);
    return ret;
}
#endif //#if ((ATBM_WIFI_PLATFORM != 10) && (ATBM_WIFI_PLATFORM != PLATFORM_AMLOGIC_S805))
```

2.4 编译

有直接编译和带参数编译两种方式

(1) 直接编译

make;make strip

编译 ble 使用 demo，执行如下命令：

make ble_demo

(2) 带参数编译

在 WIFI 驱动 22684 版本以后的驱动可以通过这种编译方式：

make platform=PLATFORM_SIGMASTAR KERDIR=/usr/lchome/yuzhihuang/Mstar/IPC_I3/linux3.18_i3/ CROSS_COMPILE=/usr/lchome/yuzhihuang/Mstar/IPC_I3/arm-linux-gnueabihf-4.8.3-201404/bin/arm-linux-gnueabihf- sys=Linux arch=arm

make platform=PLATFORM_SIGMASTAR KERDIR=/usr/lchome/yuzhihuang/Mstar/IPC_I3/linux3.18_i3/ CROSS_COMPILE=/usr/lchome/yuzhihuang/Mstar/IPC_I3/arm-linux-gnueabihf-4.8.3-201404/bin/arm-linux-gnueabihf- sys=Linux arch=arm strip

编译 ble 使用 demo，执行如下命令：

AltoBeam Inc. Confidential

make platform=PLATFORM_SIGMASTAR KERDIR=/usr/lchome/yuzhihuang/Mstar/IPC_I3/linux3.18_i3/ CROSS_COMPILE=/usr/lchome/yuzhihuang/Mstar/IPC_I3/arm-linux-gnueabihf-4.8.3-201404/bin/arm-linux-gnueabihf- sys=Linux arch=arm ble_demo

(3) 使用 build_driver.sh 脚本编译

Build_driver.sh 放在驱动根目录
需要按照如下格式添加对应的平台即可：

```
    KDIR=/usr/lchome/yuzhihuang/xm/single_IPC_small_time/111
    CC=/usr/lchome/yuzhihuang/xm/single_IPC_small_time/opt/xm_toolchain/arm-xm-linux/usr/bin/arm-xm-linux-uclibcgnueabi-
    architecture=arm
    system=Linux
    platform_soc=PLATFORM_XM530
elif [ "$1" = "AV100" ] || [ "$1" = "av100" ];then
    Compile_parameter
    KDIR=/usr/lchome/yuzhihuang/ankai/Linux/anykaAV100/AnyCloud39AV100_AK3918AV100_SDK_V1.02/AK3918AV100_SDK_V1.02/os/bd_cfg80211
    CC=/usr/lchome/yuzhihuang/ankai/Linux/anykaAV100/AnyCloud39AV100_AK3918AV100_SDK_V1.02/AK3918AV100_SDK_V1.02/tools/arm-anycloud-linux-uclibcgnueabi/bin/arm-anycloud-li
    architecture=arm
    system=Linux
    platform_soc=PLATFORM_AV100
elif [ "$1" = "37E" ] || [ "$1" = "37e" ];then
    KDIR=/usr/lchome/yuzhihuang/ankai/Linux/anyka37E/AK37E_SDK_V1.01_1/os/bd_cfg80211
    CC=/usr/lchome/yuzhihuang/ankai/Linux/anyka37E/AK37E_SDK_V1.01_1/tools/arm-anykav500-linux-uclibcgnueabi/bin/arm-anykav500-linux-uclibcgnueabi-
    architecture=arm
    system=Linux
    platform_soc=PLATFORM_37E
elif [ "$1" = "A31" ] || [ "$1" = "a31" ];then
    KDIR=/usr/lchome/yuzhihuang/allWinner/BJ_A31/linux-3.3/
    CC=/usr/lchome/yuzhihuang/allWinner/BJ_A31/tools/toolschain/gcc-linaro/bin/arm-linux-gnueabi-
    architecture=arm
    system=Linux
    platform_soc=PLATFORM_A31
elif [ "$1" = "anba" ];then
    KDIR=/usr/lchome/lishicheng/ambarella/cv2x_linux_pure_sdk_3.0/ambarella/out/cv25_hazelnut/kernel/linux-5.4_cv25
        CC=/usr/lchome/lishicheng/ambarella/linaro-aarch64-2020.09-gcc10.2-linux5.4/bin/aarch64-linux-gnu-
        architecture=arm64
        system=Linux
        platform_soc=PLATFORM_ANBA

else
    echo "$1"
    help
    return -1;
fi
```

参数说明：
KDIR: 内核路径
CC: 交叉编译器
architecture: 平台架构
system: 使用的系统
platform_soc: 平台宏
脚本编译使用的是 Makefile_SZ 文件，需要在该文件配置上末尾配置上平台宏对应的 ID

```
ATBM_WIFI__EXT_CCFLAGS ?= -DATBM_WIFI_PLATFORM=8
endif
ifeq ($(platform),PLATFORM_CDLINUX)
export
ATBM_WIFI__EXT_CCFLAGS ?= -DATBM_WIFI_PLATFORM=12
endif
ifeq ($(platform),PLATFORM_AMLOGIC_S805)
export
ATBM_WIFI__EXT_CCFLAGS ?= -DATBM_WIFI_PLATFORM=13
endif

ifeq ($(platform),PLATFORM_SIGMASTAR)
export
ATBM_WIFI__EXT_CCFLAGS ?= -DATBM_WIFI_PLATFORM=18
endif

ifeq ($(platform),PLATFORM_INGENICT31)
export
ATBM_WIFI__EXT_CCFLAGS ?= -DATBM_WIFI_PLATFORM=22
endif
ifeq ($(platform),PLATFORM_INGENICT41)
export
ATBM_WIFI__EXT_CCFLAGS ?= -DATBM_WIFI_PLATFORM=23
endif


ifeq ($(ATBM_BUILD_IN_KERNEL),)

obj-m += hal_apollo/
export ATBM_BUILD_IN_KERNEL=y

else

export
include $(src)/Makefile.build.kernel
```

AltoBeam Inc. Confidential Document

执行：./build_driver.sh 是 help 说明：

```
[yuzhihuang@ubuntu_sz_2nd bj_svn]$ ./build_driver.sh
platform:
        SIGMASTAR       :313E(3,18,30),325(4.9.84),335(4.9.84),377(5.10.61)
        ANYKA           :AV100(4.4),37E(4.4)
        FULLHAN         :FH8856(4.9.129)
        XIONMAI         :XM530(3.10.103)
        ALLWINNER       :A31(3.3.0)
        ANBA            :anba(5.4)
        ITOP            :4412(5.3.18)
compiler demo: If empty, compile all by default
        ble_demo : Compile only ble_demo,Be used for BLE set Master&Slave
        ble_coex : Compile only ble_coex ,For WIFI BLE coexistence
such as test on SIGMASTAR 313E platform
If you want to compile the whole thing please run :
        build_driver.sh 313E

If you only compile ble_demo:
        build_driver.sh 313E ble_demo

if other platform ,please run:
        build_driver.sh other KERDIR=kernel_path CROSS_COMPILE=Compiler_path/compiler sys=soc_run_sys arch=system_architecture plat
orm=PLATFORM_OTHER
[yuzhihuang@ubuntu_sz_2nd bj_svn]$
```

编译执行：

```
./build_driver.sh 313e
```

会完整的编译 WIFI 驱动 ， BLE 驱动，cfg80211 驱动，ble_demo & ble_coex 工具

如果需要单独编译 ble_demo：

```
./build_driver.sh 313e ble_demo
```

会生成 3 个工具：

```
-rwxr-xr-x 1 yuzhihuang software  7868  4月 19 15:00 atbm_cli
-rwxr-xr-x 1 yuzhihuang software 12928  4月 19 15:00 atbm_tool
-rwxr-xr-x 1 yuzhihuang software 21968  4月 19 15:00 ble_smt
```

用于配置 BLE 的 Mstar/Slave 功能。

如果需要单独编译 ble_coex:

```
./build_driver.sh 313e ble_coex
```

ble_coex 是 wifi ble 共存的调试配置工具。

2.5 驱动所在的目录

编译完成会将所生成的 ko 放在 driver_install 目录下

```
[yuzhihuang@ubuntu_sz_2nd bj_svn]$ ls -l driver_install/
total 1568
-rw-r--r-- 1 yuzhihuang software 730288  3月 15 16:34 ATBM606x_wifi_usb.ko
-rw-r--r-- 1 yuzhihuang software 515096  3月 15 16:34 ble_atbm60xx.ko
-rwxr-xr-x 1 yuzhihuang software  21968  3月 15 16:34 ble_smt
-rw-r--r-- 1 yuzhihuang software 329780  3月 15 16:34 cfg80211.ko
[yuzhihuang@ubuntu_sz_2nd bj_svn]$
```

cfg80211.ko : ATBM WIFI 自带的 CFG80211

ATBM606x_wifi_usb.ko : WIFI6 驱动

ble_atbm60xx.ko : BLE 驱动

ble_smt : BLE 控制 demo

AltoBeam Inc. Confidential Document

3 驱动放置在内核中的编译方法

3.1 将驱动放置在内核中

进入内核目录下的 drivers/net/wireless/子目录修改 Makefile 和 Kconfig 文件

```
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
fugui@ubuntu200:/wifi_prj/staff/fugui/platform/iTop4412_Kernel_3.0$ cd drivers/net/wireless/
fugui@ubuntu200:/wifi_prj/staff/fugui/platform/iTop4412_Kernel_3.0/drivers/net/wireless$
```

a) 修改 Makefile

```
obj-$(CONFIG_HOSTAP)            += hostap/
obj-$(CONFIG_B43)               += b43/
obj-$(CONFIG_B43LEGACY)         += b43legacy/
obj-$(CONFIG_ZD1211RW)          += zd1211rw/
obj-$(CONFIG_RTL8180)           += rtl818x/
obj-$(CONFIG_RTL8187)           += rtl818x/
obj-$(CONFIG_RTLWIFI)           += rtlwifi/
obj-$(CONFIG_ATBM_WIRELESS)     += atbm_wifi_40M/

# 16-bit wireless PCMCIA client drivers
obj-$(CONFIG_PCMCIA_RAYCS)      += ray_cs.o
obj-$(CONFIG_PCMCIA_WL3501)     += wl3501_cs.o
```

b) 修改 Kconfig

```
source "drivers/net/wireless/rt2x00/Kconfig"
source "drivers/net/wireless/rtlwifi/Kconfig"
source "drivers/net/wireless/wl1251/Kconfig"
source "drivers/net/wireless/wl12xx/Kconfig"
source "drivers/net/wireless/zd1211rw/Kconfig"
source "drivers/net/wireless/mwifiex/Kconfig"
source "drivers/net/wireless/atbm_wifi_40M/Kconfig"

menuconfig MTK_WIRELESS_SOLUTION
        bool "MTK wireless chip configuration"
        help
          "enable/disable and config MTK wireless solution"

if MTK_WIRELESS_SOLUTION
source "drivers/net/wireless/combo_mt66xx/Kconfig"
source "drivers/net/wireless/mt5931/Kconfig"
endif # MTK_WIRELESS_SOLUTION
endif # WLAN
```

修改完 Makefile 和 Kconfig 文件后回到内核顶层目录

将 atbm_wifi6_40M_sdk 的驱动源码目录复制到内核目录下的 drivers/net/wirelesee/

c) 修改 atbm_wifi6_40M_sdk 中的 Makefile,指定相关平台选择，默认指定为 platform_other，

PS：注意#PLATFORM_OTHER 20 ，这个值用户自定义，不要和已定义的冲突即可

```
Makefile
28 #PLATFORM_OTHER                20
29 export
30 platform ?= PLATFORM_OTHER
31 #Android
32 #Linux
33 sys ?= Linux
34 #arch:arm or arm64
35 arch ?= arm
36 #export
37 #ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=$(platform)
38
39 ifeq ($(CUSTOMER_SUPPORT_USED),y)
40 MAKEFILE_SUB ?= Makefile.build.customer
41 else
42 MAKEFILE_SUB ?= Makefile.build
43 endif
44
45 ifeq ($(KERNELRELEASE),)
46
47 ifeq ($(platform),PLATFORM_HS_IPC)
48 KERDIR:=/wifi_prj/staff/zhouzhanchao/ankai_hs_ipc/kernel/kernel_testXMFlash/kernel/
49 CROSS_COMPILE:=/wifi_prj/staff/zhouzhanchao/ankai_hs_ipc/bin/arm-2009q3/bin/arm-none-linux-gnueabi-
50 ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=17
51 arch = arm
```

选择平台

选择系统及架构

在同文件在底下，需要修改 PLATFORM_OTHER 的值

```
331 ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=12
332 endif
333 ifeq ($(platform),PLATFORM_AMLOGIC_S805)
334 export
335 ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=13
336 endif
337 ifeq ($(platform),PLATFORM_ROCKCHIP_3229)
338 export
339 ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=10
340 endif
341 ifeq ($(platform),PLATFORM_XIONGMAI)
342 export
343 ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=21
344 endif
345 ifeq ($(platform),PLATFORM_OTHER)
346 export
347 ATBM_WIFI__EXT_CCFLAGS = -DATBM_WIFI_PLATFORM=20
348 endif
349 export
350 include $(src)/Makefile.build.kernel
351 endif
```

d) 通过 make menuconfig 配置 atbm_wifi 驱动支持的相关配置

➔ 请参考文档的【一.2)】：配置驱动。

e) 通过平台相关编译方式编译得到 atbm 的驱动 ko 文件。

f) 编译出来的驱动有点大 需要 strip 缩小体积

Arm-linux-xxx-strip --strip-debug atbm_wifixxx.ko

4 出错调试信息&解决

4.1 Make menuconfig 报错

```
make[1]: execvp: ./lxdialog/check-lxdialog.sh: Permission denied
cc -Wall -Wmissing-prototypes -Wstrict-prototypes -O2 -fomit-frame-pointer  -DLOCALE   -c -o mconf.o mconf.c
cc -Wall -Wmissing-prototypes -Wstrict-prototypes -O2 -fomit-frame-pointer  -DLOCALE   -c -o zconf.tab.o zconf.tab.c
In file included from zconf.tab.c:2503:0:
menu.c: In function 'get_symbol_str':
menu.c:561:18: warning: 'jump' may be used uninitialized in this function [-Wmaybe-uninitialized]
     jump->offset = r->len - 1;
                  ^
menu.c:515:19: note: 'jump' was declared here
  struct jump_key *jump;
                   ^
cc -Wall -Wmissing-prototypes -Wstrict-prototypes -O2 -fomit-frame-pointer  -DLOCALE   -c -o lxdialog/checklist.o lxdialog/checklist.c
cc -Wall -Wmissing-prototypes -Wstrict-prototypes -O2 -fomit-frame-pointer  -DLOCALE   -c -o lxdialog/inputbox.o lxdialog/inputbox.c
cc -Wall -Wmissing-prototypes -Wstrict-prototypes -O2 -fomit-frame-pointer  -DLOCALE   -c -o lxdialog/menubox.o lxdialog/menubox.c
cc -Wall -Wmissing-prototypes -Wstrict-prototypes -O2 -fomit-frame-pointer  -DLOCALE   -c -o lxdialog/textbox.o lxdialog/textbox.c
cc -Wall -Wmissing-prototypes -Wstrict-prototypes -O2 -fomit-frame-pointer  -DLOCALE   -c -o lxdialog/util.o lxdialog/util.c
cc -Wall -Wmissing-prototypes -Wstrict-prototypes -O2 -fomit-frame-pointer  -DLOCALE   -c -o lxdialog/yesno.o lxdialog/yesno.c
cc -o mconf mconf.o zconf.tab.o lxdialog/checklist.o lxdialog/inputbox.o lxdialog/menubox.o lxdialog/textbox.o lxdialog/util.o lxdialog/yesno.o
mconf.o: In function `show_help':
mconf.c:(.text+0x7b4): undefined reference to `stdscr'
lxdialog/checklist.o: In function `print_arrows':
checklist.c:(.text+0x2c): undefined reference to `wmove'
checklist.c:(.text+0x4c): undefined reference to `acs_map'
checklist.c:(.text+0x54): undefined reference to `waddch'
checklist.c:(.text+0x66): undefined reference to `waddnstr'
checklist.c:(.text+0x76): undefined reference to `wmove'
checklist.c:(.text+0x9f): undefined reference to `acs_map'
checklist.c:(.text+0xa7): undefined reference to `waddch'
checklist.c:(.text+0xe3): undefined reference to `acs_map'
```

原因是系统缺少一些库：

```
sudo apt-get install libncurses5
sudo apt-get install libncurses5-dev
```

AltoBeam Confidential Document

4.2 编译出错

在编译驱动时，有可能出现编译限制等级较为严格导致出错。

```
  CC [M]  drivers/net/wireless/atbm_HS_svn950/hal_apollo/pm.o
drivers/net/wireless/atbm_HS_svn950/hal_apollo/atbm_platform.c:90:9: note: #pragma message: xunwei
 #pragma message(PLATFORMINF)
         ^
drivers/net/wireless/atbm_HS_svn950/hal_apollo/atbm_platform.c:403:2: error: implicit declaration of function 'EXYNOS4_GPX2' [-Werror=implicit-function-declaration]
  .irq_gpio = EXYNOS4_GPX2(4),
  ^
drivers/net/wireless/atbm_HS_svn950/hal_apollo/atbm_platform.c:403:2: error: initializer element is not constant
drivers/net/wireless/atbm_HS_svn950/hal_apollo/atbm_platform.c:403:2: error: (near initialization for 'platform_data.irq_gpio')
drivers/net/wireless/atbm_HS_svn950/hal_apollo/atbm_platform.c:404:2: error: implicit declaration of function 'EXYNOS4_GPC1' [-Werror=implicit-function-declaration]
  .power_gpio = EXYNOS4_GPC1(1),
  ^
drivers/net/wireless/atbm_HS_svn950/hal_apollo/atbm_platform.c:404:2: error: initializer element is not constant
drivers/net/wireless/atbm_HS_svn950/hal_apollo/atbm_platform.c:404:2: error: (near initialization for 'platform_data.power_gpio')
cc1: some warnings being treated as errors
make[5]: *** [drivers/net/wireless/atbm_HS_svn950/hal_apollo/atbm_platform.o] Error 1
make[5]: *** Waiting for unfinished jobs....
drivers/net/wireless/atbm_HS_svn950/hal_apollo/hwio_usb.c: In function 'atbm_before_load_firmware':
```

修改 kernel/Makefile

```
KBUILD_CFLAGS   += $(call cc-option,-Wno-stringop-truncation)
KBUILD_CFLAGS   += $(call cc-option,-Wno-sizeof-pointer-memaccess)
KBUILD_CFLAGS   += $(call cc-option,-Wno-array-bounds)
KBUILD_CFLAGS   += $(call cc-option,-Wno-packed-not-aligned)

# disable warnings in gcc 7.2.1
#KBUILD_CFLAGS   += $(call cc-option,-Wno-switch-unreachable)
#KBUILD_CFLAGS   += $(call cc-option,-Wno-misleading-indentation)

# use the deterministic mode of AR if available
KBUILD_ARFLAGS := $(call ar-option,D)

#KBUILD_CFLAGS   += -Werror

include scripts/Makefile.kasan
include scripts/Makefile.extrawarn
include scripts/Makefile.ubsan

# Add any arch overrides and user supplied CPPFLAGS, AFLAGS and CFLAGS as
# last assignments
KBUILD_CPPFLAGS += $(ARCH_CPPFLAGS) $(KCPPFLAGS)
KBUILD_AFLAGS   += $(ARCH_AFLAGS)   $(KAFLAGS)
KBUILD_CFLAGS   += $(ARCH_CFLAGS)   $(KCFLAGS)

# Use --build-id when available.
LDFLAGS_BUILD_ID = $(patsubst -Wl$(comma)%,%,\
                          $(call cc-ldoption, -Wl$(comma)--build-id,)
KBUILD_LDFLAGS_MODULE += $(LDFLAGS_BUILD_ID)
```

如果注释上面的宏还不行的话，就需要按照下面的一个个修改。

```
                -include $(srctree)/include/linux/kconfig.h

KBUILD_CPPFLAGS := -D__KERNEL__

KBUILD_CFLAGS   := -Wall -Wundef -Wstrict-prototypes -Wno-trigraphs \
                   -fno-strict-aliasing -fno-common \
                   -Wno-format-security \
                   -fno-delete-null-pointer-checks
#-Werror-implicit-function-declaration
KBUILD_AFLAGS_KERNEL :=
KBUILD_CFLAGS_KERNEL :=
```

类似警告导致 error 的问题，类似修改。

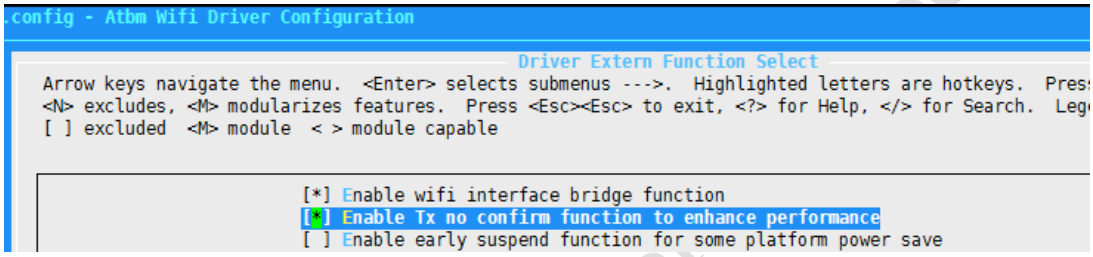
AltoBeam Confidential Document

4.3 加载出错

(1) NO_CONFIRM 宏没配置对导致出错

```
:CAPABILITIES_SHARE_CRYSTAL        [0]
:CAPABILITIES_HW_CHECKSUM          [0]
:CAPABILITIES_SINGLE_CHANNEL_MULRX [1]
:CAPABILITIES_CFO_DCXO_CORRECTION  [0]
:LMAC SET CAPABILITIES_NO_CONFIRM <ERROR>
-[ cut here ]------------
at bfd62b14 [verbose debug info unavailable]
ror: Oops - BUG: 0 [#1] PREEMPT SMP THUMB2
```

解决办法需要在一开始配置驱动时候打开对应的宏，如果打开了就给关闭。



4.4 扫描 AP 个数少

(1) 扫描状态返回-110

Log 如下图,这种可能是内核做了微小的修改以后没有再重新编译驱动以后导致的。

一般是修改 CONFIG_HZ 这个参数的值。

```
V380-linux# iwlist p2p0 scan | grep SSID
[   43.167634] [atbm_log]:atbm_hw_scan:if_id(1)
[   43.171134] [atbm_log]:atbm_hw_scan:scan, delay suspend
[   43.178162] [atbm_log]:scan start band(0),(14)
[   43.757991] [atbm_log]:Timeout waiting for scan complete notification.
[   43.763360] [atbm_log]:wsm_stop_scan_confirm 0 wait_complete 1
[   43.771316] [atbm_log]:atbm_scan_work:end(1)
[   43.774466] [atbm_log]:if_id = -1
[   43.774533] [atbm_log]:if_id = -1
[   43.774580] [atbm_log]:if_id = -1
[   43.777490] [atbm_log]:hw_priv->scan.status -110
                    ESSID:"B"
                    ESSID:"macro-video"
                    ESSID:"HUAWEI-10EC3B_Wi-Fi5"
                    ESSID:"YanFa-06090"
                    ESSID:"HUAWEI-10EC3B"
                    ESSID:""
                    ESSID:"YLGJ"
                    ESSID:"360A"
                    ESSID:""
                    ESSID:"Xiaomi_F2D4"
                    ESSID:"360_123"
                    ESSID:"gongcheng"
                    ESSID:"wifi"
                    ESSID:"ceshi02"
                    ESSID:"B_Wi-Fi5"
                    ESSID:"DIRECT-273F1225"
                    ESSID:"MV15106437"
V380-linux# [8373bf5a9d5b35b15c6f46963d89c75d]
```

Confidential Document

（2） 扫描状态正常但是扫描的 AP 数量少，并且发现前几个信道的 ap 很少或者没有

出现这种问题先查一下 cfg80211 的配置，在内核的 net/wireless/scan.c 查看如下参数配置：

```
#define IEEE80211_SCAN_RESULT_EXPIRE (15 * HZ)
```

正常 IEEE80211_SCAN_RESULT_EXPIRE 配置为 15 * HZ 或者 30 * HZ,太短会导致扫描到的 ap 个数少

4.5 添加详细的反汇编信息方法

需要在 Makefile.build.kernel 里面添加上-g 编译参数

```
#########################################################################
#               Makefile For Kernel
#########################################################################

NOSTDINC_FLAGS := -I$(src)/include/ \
        -include $(src)/include/linux/compat-2.6.h \
        -DCOMPAT_STATIC


KBUILD_CFLAGS += -Wno-error \
        -Wno-error=vla \
        -Wno-error=unused-function -g
```

这样子编译出来的驱动，执行：

```
objdump -S atbm603x_wifi.ko > atbm603x_wifi.s
```

信息就会很详细

```
Disassembly of section .text:

00000000 <atbm_timer_handle>:
#if (LINUX_VERSION_CODE >= KERNEL_VERSION(4, 14, 0))
static inline void atbm_timer_handle(struct timer_list *in_timer)
#else
static inline void atbm_timer_handle(unsigned long data)
#endif
{
       0:       e1a0c00d        mov     ip, sp
       4:       e92dd800        push    {fp, ip, lr, pc}
       8:       e24cb004        sub     fp, ip, #4
#if (LINUX_VERSION_CODE >= KERNEL_VERSION(4, 14, 0))
        struct atbm_timer_list *atbm_timer = from_timer(atbm_timer, in_timer, timer);
#else
        struct atbm_timer_list *atbm_timer = (struct atbm_timer_list *)data;
#endif
        BUG_ON(atbm_timer->function == NULL);
       c:       e590301c        ldr     r3, [r0, #28]
      10:       e3530000        cmp     r3, #0
      14:       0a000002        beq     24 <atbm_timer_handle+0x24>
        atbm_timer->function(atbm_timer->data);
      18:       e5900020        ldr     r0, [r0, #32]
      1c:       e12fff33        blx     r3
      20:       e89da800        ldm     sp, {fp, sp, pc}
      24:       e7f001f2        .word   0xe7f001f2

00000028 <ieee80211_tasklet_handler>:
              BSS_CHANGED_ERP_PREAMBLE |
              BSS_CHANGED_ERP_SLOT;
}

static void ieee80211_tasklet_handler(unsigned long data)
```

如果编译的时候没有添加-g 参数或者驱动经过了 strip 那么执行：

```
objdump -S atbm603x_wifi.ko > atbm603x_wifi.s
```

可读性就比较差。

```
Disassembly of section .text:

00000000 <atbm_timer_handle>:
       0:       e1a0c00d        mov     ip, sp
       4:       e92dd800        push    {fp, ip, lr, pc}
       8:       e24cb004        sub     fp, ip, #4
       c:       e590301c        ldr     r3, [r0, #28]
      10:       e3530000        cmp     r3, #0
      14:       0a000002        beq     24 <atbm_timer_handle+0x24>
      18:       e5900020        ldr     r0, [r0, #32]
      1c:       e12fff33        blx     r3
      20:       e89da800        ldm     sp, {fp, sp, pc}
      24:       e7f001f2        .word   0xe7f001f2

00000028 <ieee80211_tasklet_handler>:
      28:       e1a0c00d        mov     ip, sp
      2c:       e92ddff0        push    {r4, r5, r6, r7, r8, r9, sl, fp, ip, lr, pc}
      30:       e24cb004        sub     fp, ip, #4
      34:       e24dd024        sub     sp, sp, #36     ; 0x24
      38:       e3a03000        mov     r3, #0
      3c:       e24b5038        sub     r5, fp, #56     ; 0x38
      40:       e1a07000        mov     r7, r0
      44:       e50b3030        str     r3, [fp, #-48]  ; 0xffffffd0
      48:       e50b5038        str     r5, [fp, #-56]  ; 0xffffffc8
      4c:       e50b5034        str     r5, [fp, #-52]  ; 0xffffffcc
      50:       e10f0000        mrs     r0, CPSR
      54:       e3802080        orr     r2, r0, #128    ; 0x80
      58:       e121f002        msr     CPSR_c, r2
      5c:       e3a02001        mov     r2, #1
      60:       e50b3044        str     r3, [fp, #-68]  ; 0xffffffbc
      64:       e1a06003        mov     r6, r3
      68:       e2873e1a        add     r3, r7, #416    ; 0x1a0
      6c:       e50b3048        str     r3, [fp, #-72]  ; 0xffffffb8
```

4.6 编译的时候显示详细的编译信息

```
make V=1
```

```
bm_wifi/BJ_SVN/2073/AtbmWifi_Driver_SVN2073_LMAC13625_20220114/modules.order
make -f /usr/lchome/yuzhihuang/ankai/Linux/anyka37E/AK37E_SDK_V1.01_1/os/kernel/scripts/Makefile.modpost
  find /usr/lchome/yuzhihuang/Mstar/325/atbm_wifi/BJ_SVN/2073/AtbmWifi_Driver_SVN2073_LMAC13625_20220114/.tmp_versions -name '*.mod' | xargs -r grep -h '\.ko$' | sort -u | sed 's/\.ko$
/.o/' | scripts/mod/modpost   -i ./Module.symvers -I /usr/lchome/yuzhihuang/Mstar/325/atbm_wifi/BJ_SVN/2073/AtbmWifi_Driver_SVN2073_LMAC13625_20220114/Module.symvers  -o /usr/lchome/yu
zhihuang/Mstar/325/atbm_wifi/BJ_SVN/2073/AtbmWifi_Driver_SVN2073_LMAC13625_20220114/Module.symvers -S -E -w  -s -T -
  /usr/lchome/yuzhihuang/ankai/Linux/anyka37E/AK37E_SDK_V1.01_1/tools/arm-anykav500-linux-uclibcgnueabi/bin/arm-anykav500-linux-uclibcgnueabi-gcc -Wp,-MD,/usr/lchome/yuzhihuang/Mstar/3
25/atbm_wifi/BJ_SVN/2073/AtbmWifi_Driver_SVN2073_LMAC13625_20220114/hal_apollo/.atbm603x_37E_HT20.mod.o.d -I/usr/lchome/yuzhihuang/Mstar/325/atbm_wifi/BJ_SVN/2073/AtbmWifi_Driver_SVN20
73_LMAC13625_20220114/include/ -include /usr/lchome/yuzhihuang/Mstar/325/atbm_wifi/BJ_SVN/2073/AtbmWifi_Driver_SVN2073_LMAC13625_20220114/include/linux/compat-2.6.h -DCOMPAT_STATIC -g
-I/usr/lchome/yuzhihuang/ankai/Linux/anyka37E/AK37E_SDK_V1.01_1/os/kernel/arch/arm/include -Iarch/arm/include/generated/uapi -Iarch/arm/include/generated  -I/usr/lchome/yuzhihuang/anka
i/Linux/anyka37E/AK37E_SDK_V1.01_1/os/kernel/include -Iinclude -I/usr/lchome/yuzhihuang/ankai/Linux/anyka37E/AK37E_SDK_V1.01_1/os/kernel/arch/arm/include/uapi -Iarch/arm/include/gener
ted/uapi -I/usr/lchome/yuzhihuang/ankai/Linux/anyka37E/AK37E_SDK_V1.01_1/os/kernel/include/uapi -Iinclude/generated/uapi -include /usr/lchome/yuzhihuang/ankai/Linux/anyka37E/AK37E_SDK_
V1.01_1/os/kernel/include/linux/kconfig.h   -I/usr/lchome/yuzhihuang/Mstar/325/atbm_wifi/BJ_SVN/2073/AtbmWifi_Driver_SVN2073_LMAC13625_20220114 -D__KERNEL__ -mlittle-endian   -I/usr/l
home/yuzhihuang/ankai/Linux/anyka37E/AK37E_SDK_V1.01_1/os/kernel/arch/arm/mach-anycloud/include -Wall -Wundef -Wstrict-prototypes -Wno-trigraphs -fno-strict-aliasing -fno-common -Werro
r-implicit-function-declaration -Wno-format-security -std=gnu89 -fno-PIE -fno-dwarf2-cfi-asm -fno-omit-frame-pointer -mapcs -mno-sched-prolog -fno-ipa-sra -mabi=aapcs-linux -mno-thumb-
interwork -mfpu=vfp -marm -D__LINUX_ARM_ARCH__=5 -march=armv5te -mtune=arm9tdmi -msoft-float -Uarm -fno-delete-null-pointer-checks -Wno-maybe-uninitialized -O2 --param=allow-store-data
-races=0 -DCC_HAVE_ASM_GOTO -Wframe-larger-than=4096 -fno-stack-protector -Wno-unused-but-set-variable -fno-omit-frame-pointer -fno-optimize-sibling-calls -fno-var-tracking-assignments
 -Wdeclaration-after-statement -Wno-pointer-sign -fno-strict-overflow -fno-merge-all-constants -fmerge-constants -fno-stack-check -fconserve-stack -Werror=implicit-int -Werror=strict-p
rototypes -Werror=date-time -Wno-error -Wno-error=vla -Wno-error=unused-function -g  -D"KBUILD_STR(s)=#s" -D"KBUILD_BASENAME=KBUILD_STR(atbm603x_37E_HT20.mod)"  -D"KBUILD_MODNAME=KBUI
D_STR(atbm603x_37E_HT20)" -DMODULE  -c -o /usr/lchome/yuzhihuang/Mstar/325/atbm_wifi/BJ_SVN/2073/AtbmWifi_Driver_SVN2073_LMAC13625_20220114/hal_apollo/atbm603x_37E_HT20.mod.o /usr/lch
me/yuzhihuang/Mstar/325/atbm_wifi/BJ_SVN/2073/AtbmWifi_Driver_SVN2073_LMAC13625_20220114/hal_apollo/atbm603x_37E_HT20.mod.c
  /usr/lchome/yuzhihuang/ankai/Linux/anyka37E/AK37E_SDK_V1.01_1/tools/arm-anykav500-linux-uclibcgnueabi/bin/arm-anykav500-linux-uclibcgnueabi-ld -EL -r  -T /usr/lchome/yuzhihuang/ankai
/Linux/anyka37E/AK37E_SDK_V1.01_1/os/kernel/scripts/module-common.lds --build-id  -o /usr/lchome/yuzhihuang/Mstar/325/atbm_wifi/BJ_SVN/2073/AtbmWifi_Driver_SVN2073_LMAC13625_20220114/H
al_apollo/atbm603x_37E_HT20.ko /usr/lchome/yuzhihuang/Mstar/325/atbm_wifi/BJ_SVN/2073/AtbmWifi_Driver_SVN2073_LMAC13625_20220114/hal_apollo/atbm603x_37E_HT20.o /usr/lchome/yuzhihuang/M
star/325/atbm_wifi/BJ_SVN/2073/AtbmWifi_Driver_SVN2073_LMAC13625_20220114/hal_apollo/atbm603x_37E_HT20.mod.o
make[2]: Leaving directory '/usr/lchome/yuzhihuang/ankai/Linux/anyka37E/AK37E_SDK_V1.01_1/os/bd'
mkdir -p /usr/lchome/yuzhihuang/Mstar/325/atbm_wifi/BJ_SVN/2073/AtbmWifi_Driver_SVN2073_LMAC13625_20220114/driver_install/
chmod 777 /usr/lchome/yuzhihuang/Mstar/325/atbm_wifi/BJ_SVN/2073/AtbmWifi_Driver_SVN2073_LMAC13625_20220114/driver_install/
cp hal_apollo/*.ko       /usr/lchome/yuzhihuang/Mstar/325/atbm_wifi/BJ_SVN/2073/AtbmWifi_Driver_SVN2073_LMAC13625_20220114/driver_install/
make[1]: Leaving directory '/usr/lchome/yuzhihuang/Mstar/325/atbm_wifi/BJ_SVN/2073/AtbmWifi_Driver_SVN2073_LMAC13625_20220114'
```

4.7 编译报时钟错误

原因可能是系统时间太老了，需要重启下电脑！

AltoBeam Confidential Document

```
yzh@yzh-VirtualBox:~/sigmastar_i7/kernel$ make infinity7_ssc023a_s01a_nandflash_defconfig
make: Warning: File 'makefile' has modification time 54663274 s in the future
Check Kconfigs for no newline at the end...
Extract CHIP NAME (infinity7) to '.sstar_chip.txt'
make[1]: Entering directory '/home/yzh/sigmastar_i7/kernel'
make[1]: Warning: File 'arch/arm64/Makefile' has modification time 54663275 s in the future
make[2]: Warning: File 'scripts/Makefile.host' has modification time 54663302 s in the future
  HOSTCC  scripts/basic/fixdep
make[2]: 警告：检测到时钟错误。您的创建可能是不完整的。
make[2]: Warning: File 'scripts/Makefile.host' has modification time 54663302 s in the future
  HOSTCC  scripts/kconfig/conf.o
  HOSTCC  scripts/kconfig/confdata.o
  HOSTCC  scripts/kconfig/expr.o
  LEX     scripts/kconfig/lexer.lex.c
/bin/sh: 1: flex: not found
scripts/Makefile.host:9: recipe for target 'scripts/kconfig/lexer.lex.c' failed
make[2]: *** [scripts/kconfig/lexer.lex.c] Error 127
Makefile:632: recipe for target 'infinity7_ssc023a_s01a_nandflash_defconfig' failed
make[1]: *** [infinity7_ssc023a_s01a_nandflash_defconfig] Error 2
make[1]: Leaving directory '/home/yzh/sigmastar_i7/kernel'
makefile:25: recipe for target 'infinity7_ssc023a_s01a_nandflash_defconfig' failed
make: *** [infinity7_ssc023a_s01a_nandflash_defconfig] Error 2
yzh@yzh-VirtualBox:~/sigmastar_i7/kernel$
```

4.8 编译报驱动自带的 cfg80211.ko 里面有一些 undefined

```
  MODPOST 3 modules
ERROR: "iwe_stream_add_point" [/usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211.ko] undefined!
ERROR: "wireless_send_event" [/usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211.ko] undefined!
ERROR: "wireless_nlevent_flush" [/usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211.ko] undefined!
ERROR: "iwe_stream_add_event" [/usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211.ko] undefined!
ERROR: "iwe_stream_add_value" [/usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211.ko] undefined!
make[4]: *** [modules-modpost] Error 1
```

原因是内核的 CFG80211 没有打开配置为模块

4.9 编译驱动报 cfg80211 函数 export twice Previous export was in vmlinux

```
  LD [M]  /usr/lchome/yuzhihuang/WIFI6/bj_svn/ble_host/ble_atbm60xx.o
  Building modules, stage 2.
  MODPOST 3 modules
WARNING: /usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211: 'wiphy_rfkill_set_hw_state' exported twice. Previous export was in vmlinux
WARNING: /usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211: 'cfg80211_stop_iface' exported twice. Previous export was in vmlinux
WARNING: /usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211: 'wiphy_rfkill_stop_polling' exported twice. Previous export was in vmlinux
WARNING: /usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211: 'wiphy_unregister' exported twice. Previous export was in vmlinux
WARNING: /usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211: 'wiphy_register' exported twice. Previous export was in vmlinux
WARNING: /usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211: 'cfg80211_unregister_wdev' exported twice. Previous export was in vmlinux
WARNING: /usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211: 'wiphy_rfkill_start_polling' exported twice. Previous export was in vmlinux
WARNING: /usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211: 'cfg80211_shutdown_all_interfaces' exported twice. Previous export was in vmlinux
WARNING: /usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211: 'wiphy_free' exported twice. Previous export was in vmlinux
WARNING: /usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211: 'wiphy_new_nm' exported twice. Previous export was in vmlinux
WARNING: /usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211: 'ieee80211_radiotap_iterator_init' exported twice. Previous export was in vmlinux
WARNING: /usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211: 'ieee80211_radiotap_iterator_next' exported twice. Previous export was in vmlinux
WARNING: /usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211: 'bridge_tunnel_header' exported twice. Previous export was in vmlinux
WARNING: /usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211: 'ieee80211_chandef_to_operating_class' exported twice. Previous export was in vmlinux
WARNING: /usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211: 'ieee80211_get_response_rate' exported twice. Previous export was in vmlinux
WARNING: /usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211: 'ieee80211_get_mesh_hdrlen' exported twice. Previous export was in vmlinux
WARNING: /usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211: 'cfg80211_get_p2p_attr' exported twice. Previous export was in vmlinux
WARNING: /usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211: 'ieee80211_amsdu_to_8023s' exported twice. Previous export was in vmlinux
WARNING: /usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211: 'ieee80211_ie_split_ric' exported twice. Previous export was in vmlinux
WARNING: /usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211: 'rfc1042_header' exported twice. Previous export was in vmlinux
WARNING: /usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211: 'cfg80211_classify8021d' exported twice. Previous export was in vmlinux
WARNING: /usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211: 'cfg80211_calculate_bitrate' exported twice. Previous export was in vmlinux
WARNING: /usr/lchome/yuzhihuang/WIFI6/bj_svn/wireless/cfg80211: 'ieee80211_get_hdrlen_from_skb' exported twice. Previous export was in vmlinux
```

原因是内核额 cfg80211 配置为 build in 方式，需要配置为模块方式

Inc. Confidential Document

4.10 Linux 3.10 内核同时跑 ap+sta 模式失败

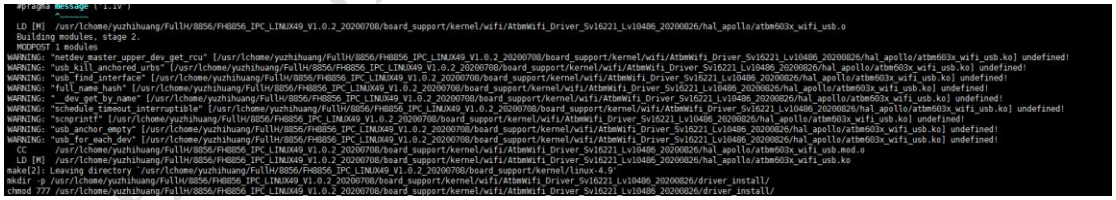
需要修改下内核的 cfg80211

```
--- net/wireless/util.c      2021-02-03 11:20:58.259603493 +0800
+++ net/wireless/util_new.c    2021-02-03 11:20:47.507602976 +0800
@@ -1316,7 +1316,13 @@


                return -EBUSY;
        }
+       if(rdev->wiphy.dev.init_name && memcmp(rdev->wiphy.dev.init_name,"ALTOBEAM",8) == 0){
+               printk("this is altobeam wifi,only support signle channel run\n");
+               break;
+       }
        if (used_channels[i] == NULL) {
                used_channels[i] = ch;
                num_different_channels++;
```

4.11 富瀚平台编译 ATBM WIFI 驱动会报一些函数 undefined

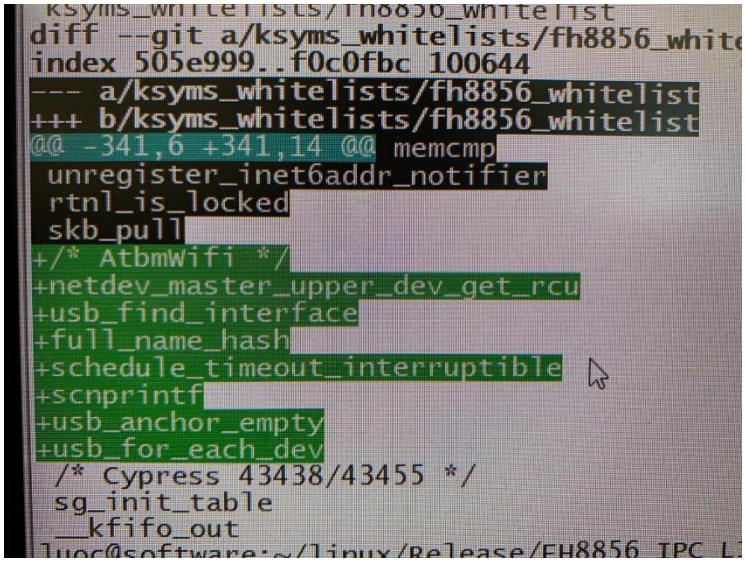
报错的 log 如下：



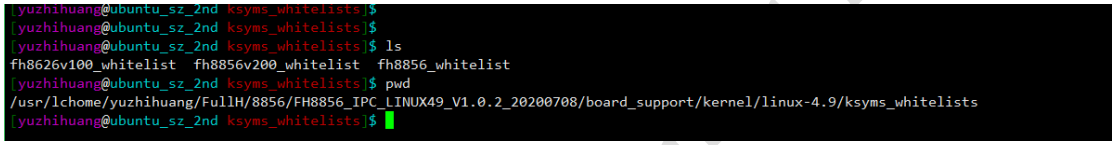
解决方法：

需要把这些符号加入内核的 fhxxxx_whilelist 文件再重新编译 kernel 和 wifi.

AltoBeam Inc. Confidential Document



实际 fhxxx_whitelist 路劲如下：



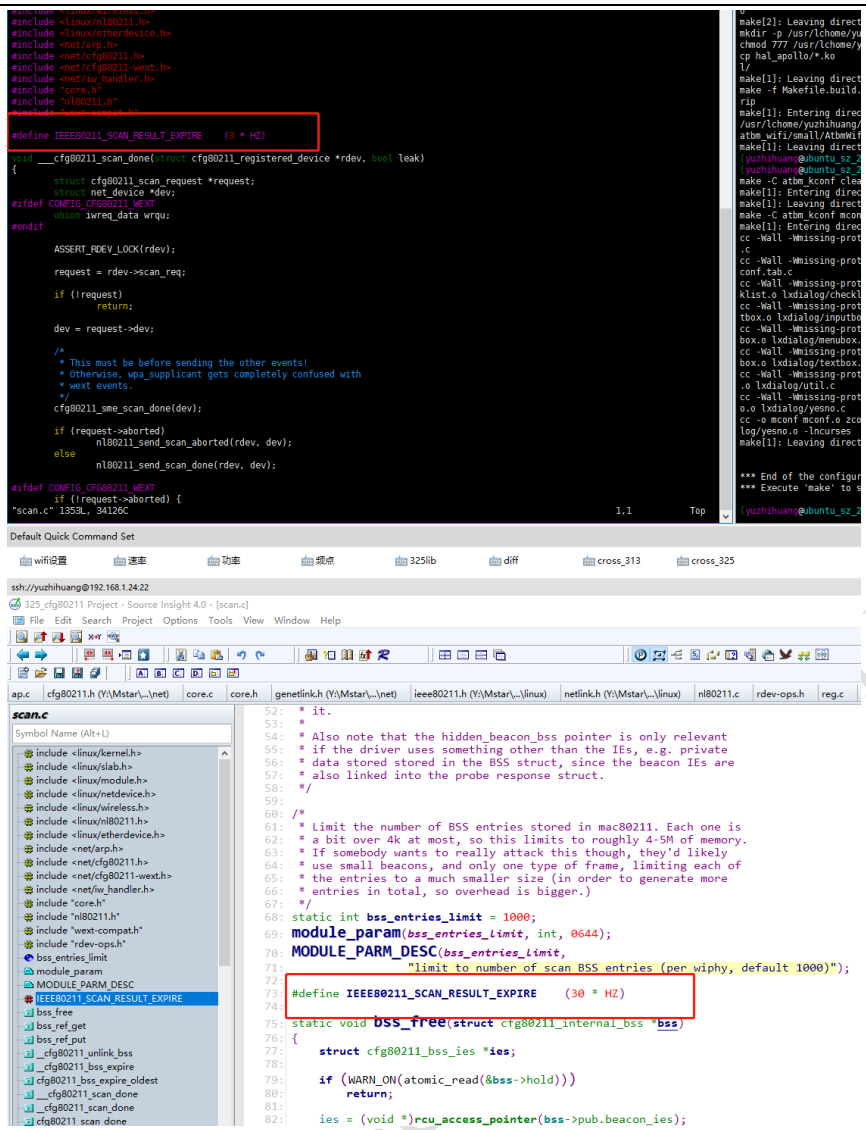
4.12 扫描不到某些信道的 AP

(1) 扫不到低信道的 ap

可以正常扫描，但是在扫描时候发现一直没有低信道的 ap，高信道的 ap 都有。

需要修改下内核的 cfg80211 的 scan.c
net/wireless/scan.c 中 IEEE80211_SCAN_RESULT_EXPIRE 宏的值由原来的 3*Hz 修改为 30*Hz

AltoBeam Inc. Confidential Document



(2) 扫不到高信道的 AP

扫描的时候会报打印：Timeout waiting for scan complete notification.
但是可以扫描到 AP 信息，但是没有高信道的 AP。

原因可能是 编译驱动的内核 和 板子里面运行的内核 不一致导致。

需要确认编译驱动的内核.config 里面的 CONFIG_HZ 是否被修改。

AltoBeam Inc. Document

```
CONFIG_NR_CPUS=4
CONFIG_HOTPLUG_CPU=y
# CONFIG_NUMA is not set
CONFIG_HOLES_IN_ZONE=y
CONFIG_HZ_100=y
# CONFIG_HZ_250 is not set
# CONFIG_HZ_300 is not set
# CONFIG_HZ_1000 is not set
CONFIG_HZ=100
CONFIG_SCHED_HRTICK=y
CONFIG_ARCH_SUPPORTS_DEBUG_PAGEALLOC=y
CONFIG_ARCH_SPARSEMEM_ENABLE=y
CONFIG_ARCH_SPARSEMEM_DEFAULT=y
CONFIG_ARCH_SELECT_MEMORY_MODEL=y
CONFIG_ARCH_FLATMEM_ENABLE=y
```

这种问题可能是内核的 CONFIG_HZ=100 被修改为 CONFIG_HZ=1000 导致的。

AltoBeam Inc. Confidential Document

AltoBeam Inc. Confidential Document

DISCLAIMER

Information in this document is provided in connection with AltoBeam products. No license, express or implied, by estoppels or otherwise, to any intellectual property rights is granted by this document. Except as provided in AltoBeam's terms and conditions of sale for such products, AltoBeam assumes no liability whatsoever, and AltoBeam disclaims any express or implied warranty, relating to sale and/or use of AltoBeam products including liability or warranties relating to fitness for a particular purpose, merchantability, or infringement of any patent, copyright or other intellectual property right.

AltoBeam may make changes to specifications and product descriptions at any time, without notice.

Designers must not rely on the absence or characteristics of any features or instructions marked "reserved" or "undefined." AltoBeam reserves these for future definition and shall have no responsibility whatsoever for conflicts or incompatibilities arising from future changes to them.

Unauthorized use of information contained herein, disclosure or distribution to any third party without written permission of AltoBeam is prohibited.

AltoBeam™ is the trademark of AltoBeam. All other trademarks and product names are properties of their respective owners.

Copyright © 2007~2025 AltoBeam, all rights reserved